CONSTRUCTOR UNIVERSITY

ADC-Aware Quantization of Large Language Models for In-Memory Computing

by

Alina Burykina

Master Thesis in Computer Science and Software Engineering

Submission: May 17, 2026 Supervisor: Prof. Dmitry Vetrov

Constructor University | Constructor Institute of Technology

Statutory Declaration

Family Name, Given/First Name	Burykina, Alina
Matriculation number	00000000
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

.....

Date, Signature

Abstract

Analog optical matrix-vector accelerators promise energy-efficient inference for large language models (LLMs), but they expose a failure mode absent from standard digital deployment: after each analog accumulation, a finite-resolution analog-to-digital converter (ADC) maps the output onto a hardware-fixed quantization lattice. This post-accumulation quantizer is especially damaging for large language models, whose channel-wise outliers make a single ADC scale difficult to use. Previous hardware-aware approaches commonly rely on retraining through the hardware model; for billion-parameter large language models, such quantization-aware training is often impractical because the original training data and compute budget are unavailable.

This thesis instead investigates whether an already trained large language model can be adapted in the post-training setting. Building on FlatQuant, an ADC-aware post-training quantization (PTQ) pipeline is introduced in which calibration is made to match analog inference by propagating ADC-quantized activations across blocks, mixing clean and ADC-aware objectives, and applying bounded diagonal scaling with staged calibration. On Llama-3.2-1B in the 4-bit weight and 4-bit activation (W4A4) setting, standard FlatQuant remains usable when the ADC is disabled, with WikiText-2 perplexity 15.41, but collapses when the ADC is enabled, reaching 2354. ADC-aware post-training quantization reduces this to 26.66, showing that transform-based calibration is necessary but plateaus far from the bfloat16 baseline of 8.68.

To close the remaining gap without full quantization-aware training, a rank-4 digital residual branch is added after the ADC, and only this low-rank correction is trained using cross-entropy and teacher Kullback–Leibler distillation. This 0.22%-parameter adaptation reduces WikiText-2/C4 ADC perplexity from $26.66/45.62$ to $\mathbf{14.03}/\mathbf{23.30}$. These results show that fixed low-resolution ADCs require algorithmic adaptation beyond digital post-training quantization, and that lightweight output-side digital correction can recover much of the accuracy lost at the analog-digital boundary.

Contents

1 Introduction

Large language models (LLMs) have become the dominant architecture for modern natural language processing (NLP), but their inference cost remains a major deployment obstacle. Transformer decoders are built primarily from large matrix–vector and matrix–matrix multiplications, and serving them requires repeatedly moving weights and activations between memory and compute units. In fact, autoregressive decoding is already memory-bandwidth-bound on modern GPUs: each forward pass reads billions of parameters to produce only a single output token, so the arithmetic intensity falls far below the hardware roofline and throughput is limited by memory bandwidth rather than compute [22]. This motivates hardware architectures that move computation closer to memory, eliminating the data movement bottleneck entirely, including in-memory computing (IMC) accelerators, where matrix–vector multiplication can be performed directly inside memory arrays rather than in a conventional digital compute core [32, 15, 4].

IMC is attractive for LLM inference because transformer layers are dominated by linear projections. In an idealized IMC pipeline, the weight matrix of a projection is stored in a memory array, an input activation vector is applied to the array, and the analog accumulation implements the dot product. However, this analog result must be converted back into a digital value before it can be consumed by the next layer, placing the analog-to-digital converter (ADC) directly on the neural network inference path. The precision and range of the ADC therefore determine how much information from the analog accumulation is preserved.

This thesis studies the interaction between transformer quantization and ADC-constrained analog inference. In a conventional digital quantization setting, the main sources of error are weight and activation quantization. In an IMC accelerator there is an additional output quantization stage: the accumulated analog result is passed through the ADC, which quantizes it to a fixed step size Δ and clips it to the ADC output range. The central difference from digital low-bit inference is that even if weights and activations are quantized well, the accumulated output can still be severely distorted by a coarse ADC lattice.

Unlike the activation and weight scales, which are adjusted during calibration, the ADC step size Δ is fixed by the analog hardware at design time and cannot be reduced by retraining or by changing the learned quantization transform. The only software-side levers available are how the signal is shaped before the ADC and how the quantized output is corrected after it. This makes ADC-constrained inference a different optimization problem from standard digital PTQ.

LLMs are a particularly challenging target for this setting. Prior work has shown that transformer activations contain large outlier channels, and that these outliers are a major obstacle to low-bit quantization [8, 31, 28, 2, 19]. Because activation scales are computed from the per-token maximum, a single large channel sets the scale for the entire activation vector. Typical activation values then receive very small integer codes, reducing the effective density of the integer dot product. In an ADC pipeline this can produce poorly occupied ADC bins or dead regions where small accumulated outputs collapse to zero. LLM activation outliers therefore affect both activation quantization and the quality of the analog output seen by the ADC.

This thesis focuses on decoder-only transformer LLMs, using Llama-3.2-1B as the main experimental target. The goal is to study whether transform-based post-training quantization (PTQ), combined with ADC-aware calibration and lightweight adaptation, can make

low-bit LLaMA inference viable under ADC-constrained analog computation. The work starts from FlatQuant, a transform-based PTQ method that learns layer-wise affine transformations to make weights and activations easier to quantize [18]. FlatQuant changes the coordinate system in which quantization is applied while leaving the linear map unchanged. The transformed activation distribution is flatter, so per-token scaling is no longer dominated by a few outlier channels.

Standard PTQ methods optimize for digital quantization and do not account for ADC output discretization as a separate source of error. Even with well-calibrated weights and activations, ADC-path perplexity remains substantially higher than no-ADC perplexity, the quality achievable with digital quantization alone. Closing this gap requires calibration that is explicitly sensitive to ADC noise and, ultimately, a lightweight correction stage applied after the ADC path. This thesis makes the following contributions:

1. ADC-constrained formulation for LLaMA PTQ. The thesis formulates LLaMA inference under a tiled ADC-constrained matrix–vector multiplication (MVM) model and implements a simulation pipeline with per-token activation quantization, per-channel weight quantization, integer accumulation, ADC output quantization, and dequantization.
2. Diagnosis of ADC-specific failure modes. The thesis separates ordinary weight and activation quantization error from ADC output quantization error using no-ADC evaluation, dead-rate, ADC bin usage, reconstruction error, and layer/projection ablations. This shows that dead zones can be caused by poor transforms, but that the remaining degradation can be dominated by ADC output resolution.
3. ADC-aware FlatQuant calibration. The thesis introduces four extensions to FlatQuant that target ADC-specific degradation: block-wise calibration with ADC propagation, an α-mixing objective, trainable per-channel diagonals, and staged training. Together these reduce INT4 ADC perplexity from 2354 (naïve baseline) to 27.6 on WikiText-2.
4. Analysis of pure PTQ limitations. Extensive ablations show that additional dead-zone penalties, aggressive PACT-style clipping, fixed Hadamard initialization, and bin-center losses are insufficient or harmful in the studied setting. Pure PTQ improves substantially but eventually reaches a plateau beyond which further calibration yields diminishing returns.
5. Residual post-ADC low-rank correction. The thesis introduces a lightweight residual correction stage on top of the frozen quantized ADC path, trained with distillation against a full-precision teacher. With only 0.22% extra parameters, this stage drops INT4 ADC perplexity from 27.6 to 14.03 on WikiText-2 and from 46.40 to 23.30 on C4, while keeping the base ADC model frozen.

2 Background and Related Work

2.1 In-Memory Computing and ADC-Constrained Inference

In-memory computing (IMC) executes matrix–vector multiplication (MVM) directly inside memory arrays, reducing the cost of data movement that dominates conventional digital inference pipelines [32, 15, 4]. In crossbar-style accelerators, neural network weights are mapped to cell conductances, the input vector is applied as an analog excitation, and

the array physics performs the accumulation. This is attractive for large neural networks because MVM is the dominant operation in both attention and MLP blocks. However, the result of the array is analog, whereas the next layer expects a digital tensor. Therefore, each analog MVM must be followed by analog-to-digital conversion (ADC).

The same analog-to-digital bottleneck appears across very different physical substrates. Memristive and phase-change-memory crossbars encode weights as cell conductances and exploit Ohm's law plus Kirchhoff's current summation for the multiply–accumulate [32, 15, 4]; photonic systems instead encode weights and activations as light intensities or phases and perform the MVM through optical interference, reading the result with photodetectors [26]. Despite the different physics, the common feature is that the analog accumulation is then sampled by a finite-resolution ADC before the next layer can consume it. Figure 1 illustrates this for the optical case: the analog signal carrying the MVM result is converted to a digital code through a photodetector front-end followed by an ADC, after which it re-enters the digital data path.

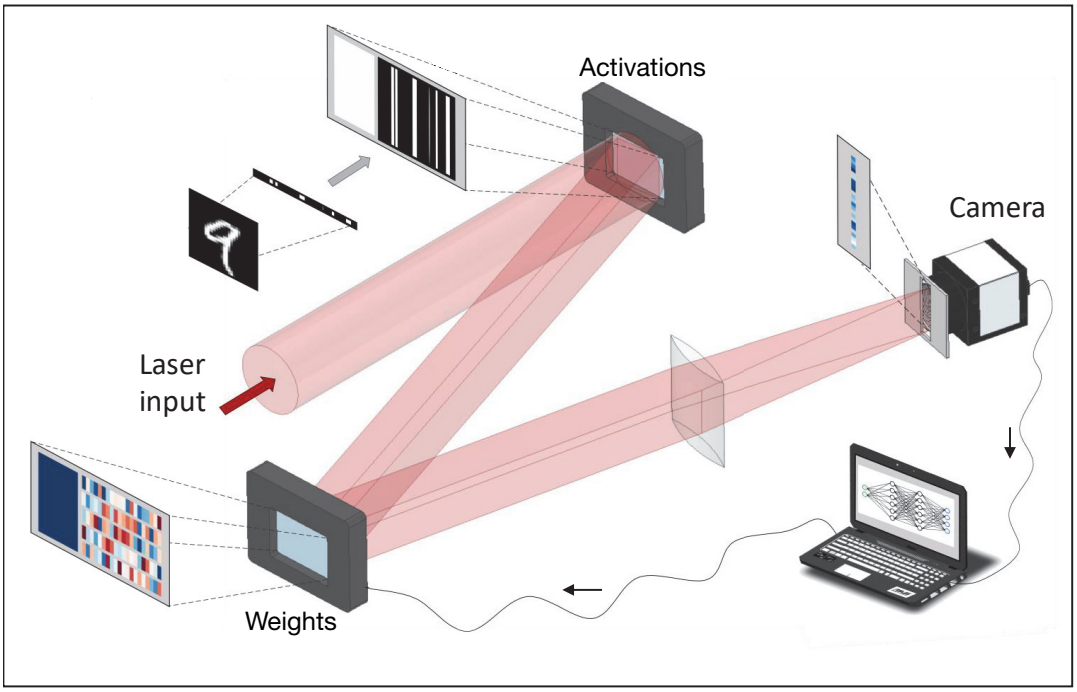

Figure 1: Physical example of an analog matrix–vector multiplication feeding an ADC. The figure, reproduced from Spall, Guo, and Lvovsky [26], shows an optical neural-network front-end: weights and activations are encoded into light fields, the inner product is computed by optical interference, and the analog accumulator output is read by a photodetector before being sampled by an ADC. The same analog-input–ADC-output structure underlies electronic in-memory computing crossbars; the mathematical model in Eqs. (6)–(7) abstracts both.

A simplified ADC-constrained inference pipeline, illustrated in Figure 2, follows the math model adopted by prior work on analog and ADC-constrained inference [27] and can be

written as

$$x_q = Q_x(x), \tag{1}$$
$$W_q = Q_w(W), \tag{2}$$
$$y_{\text{int}} = W_q x_q, \tag{3}$$
$$y_{\text{adc}} = Q_{\text{ADC}}(y_{\text{int}}), \tag{4}$$
$$\hat{y} = D_{\text{out}}\, y_{\text{adc}}, \tag{5}$$

where Q_x and Q_w are the activation and weight quantizers, Q_{ADC} denotes the output quantizer implemented by the ADC, and D_{out} is the dequantization map. In the ADC model used throughout this thesis, the analog accumulation is quantized by a coarse output lattice:

$$Q_{\text{ADC}}(u) = \Delta \cdot \text{clip}\left(\left\lfloor \frac{u}{\Delta} \right\rfloor, n_a, p_a\right), \tag{6}$$

where Δ is the ADC step size and (n_a, p_a) are the output code bounds. In a tiled IMC setting, one common hardware model yields

$$\Delta = \frac{2\, n_t\, q_x\, q_w}{2^{b_a} k}, \tag{7}$$

where n_t is the tile size, b_a is the ADC bitwidth, k is the ADC scaling factor, and q_x, q_w are the maximum activation and weight code levels. Equation (7) makes the hardware trade-off explicit: larger tiles or lower ADC precision lead to coarser output quantization.

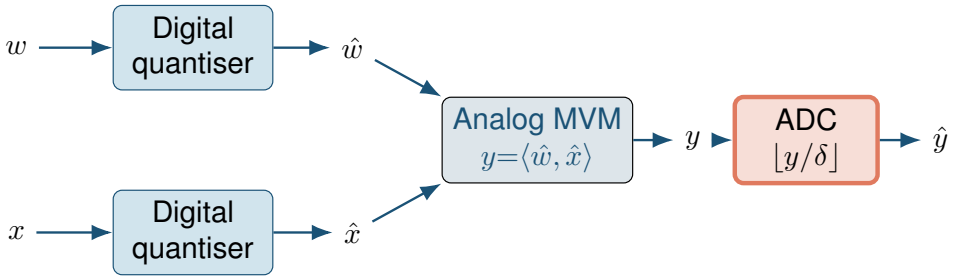

Figure 2: Math model of an analog linear layer. Activations x and weights w are first quantized digitally to $\hat{x}$ and $\hat{w}$. The analog MVM engine then computes the inner product $y = \langle \hat{w}, \hat{x} \rangle$ in continuous-valued form. Finally the ADC maps y to a discrete output $\hat{y}$ via $\lfloor y/\delta \rfloor$, with step size δ fixed by the hardware. The two distinct quantization stages (digital input, analog-to-digital output) are formalised in Eqs. (6)–(7).

This output quantization stage is qualitatively different from standard digital quantization. In purely digital low-bit inference, the dominant errors are usually input quantization, weight quantization, and approximate integer arithmetic. In IMC, even if Q_x and Q_w are well calibrated, the accumulated output can still be projected onto an insufficiently fine ADC lattice [27, 4]. Thus, the main bottleneck may shift from input-side quantization to output quantization. This distinction is central to the present thesis.

The problem becomes more severe for transformer models. Their hidden states are high-dimensional, anisotropic, and may contain rare channels with extremely large magnitude. These outliers stress the activation quantizer before the analog array and also distort the distribution of y_{int} presented to the ADC. Consequently, the question is how to quantize weights and activations and how to shape internal signals so that the analog accumulation uses the ADC range efficiently [27, 4].

2.2 Quantization of Neural Networks

Large neural networks, and large language models in particular, are bottlenecked by memory bandwidth and the arithmetic throughput of dense linear layers in floating-point precision. Quantization addresses this by representing weights and activations as low-bit integers, trading numerical accuracy for substantial reductions in model size and compute cost. In analog or IMC settings this is more than an optimization: the hardware itself operates at a fixed and often low precision, so the digital side of the pipeline must match it.

Quantization maps continuous tensors to a finite set of discrete values in order to reduce memory, bandwidth, and arithmetic cost [14, 30, 12]. In symmetric uniform quantization, a scalar $x \in \mathbb{R}$ is mapped to

$$q(x) = \mathrm{clip}\Big(\Big\lfloor \frac{x}{s} \Big\rceil, -q_{\max}, q_{\max}\Big), \qquad \hat{x} = s\, q(x), \tag{8}$$

where $s > 0$ is the quantization scale and $q_{\max} = 2^{b-1} - 1$ for a signed b-bit quantizer. Symmetric quantization is widely used for weights and signed activations because it preserves zero exactly and simplifies integer arithmetic.

In asymmetric quantization, one adds a zero-point z:

$$q(x) = \mathrm{clip}\Big(\Big\lfloor \frac{x}{s} + z \Big\rceil, q_{\min}, q_{\max}\Big), \qquad \hat{x} = s\,(q(x) - z). \tag{9}$$

This is useful when the signal distribution is non-negative or strongly shifted, as in some post-activation tensors or hardware-aware activation coding schemes [14, 30].

Quantization also differs in granularity. In per-tensor quantization, one scale is shared by the full tensor. In per-channel quantization, each output channel receives its own scale; this is particularly useful for weights because different rows of W often have very different ranges. For transformer activations, one also encounters per-token and per-group scaling, especially when the activation distribution changes strongly across tokens or heads.

Two training regimes dominate the literature. In post-training quantization (PTQ), a pre-trained floating-point model is quantized after training, usually using a small calibration set and a local reconstruction objective [33, 16, 25, 9, 18, 1]. A generic PTQ objective for layer ℓ can be written as

$$\min_{\theta_\ell^q} \sum_{x \in \mathcal{C}} \Big\| f_\ell(x; \theta_\ell) - \hat{f}_\ell(x; \theta_\ell^q) \Big\|_2^2, \tag{10}$$

where $\mathcal{C}$ is a calibration set and θ_ℓ^q are the quantized parameters or auxiliary calibration variables. PTQ is attractive because it avoids retraining the full model.

In quantization-aware training (QAT), quantization is inserted into the forward pass during training or finetuning:

$$\min_{\theta} \sum_{(x,y)} \mathcal{L}\big(y,\ f_q(x; \theta)\big), \tag{11}$$

so that the parameters adapt directly to quantization noise [7, 11, 17, 6]. Since rounding is non-differentiable, QAT commonly relies on the straight-through estimator (STE),

which uses the quantized function in the forward pass but approximates its gradient in the backward pass by

$$\frac{\partial q(x)}{\partial x} \approx 1. \tag{12}$$

Although heuristic, STE is one of the standard tools behind low-bit QAT and learned PTQ methods [7, 11].

Several refinements are especially relevant for the present work. PACT learns activation clipping thresholds [7]; LSQ learns the step size itself [11]; AdaRound learns non-trivial rounding decisions in PTQ [21]; and QDrop introduces stochastic activation dequantization during calibration to stabilize extremely low-bit PTQ [29]. These techniques all point to the same theme: in low-bit settings, choosing the bitwidth is not enough; one must also decide how the signal is clipped, scaled, rounded, and propagated.

2.3 Challenges of Quantizing Transformer Models

Transformer language models are difficult to quantize. A major reason is the presence of activation outliers. Dettmers et al. introduced LLM.int8() [8], the first work to show that hidden states in transformers contain rare but very large feature values, and that preserving these outlier dimensions matters even when the rest of the model is quantized aggressively. The same phenomenon motivates later methods such as SmoothQuant, Outlier Suppression+, QuaRot, SpinQuant, and FlatQuant [31, 28, 2, 19, 18].

Figure 3 illustrates this on a single decoder block of Llama-3.2-1B: one hidden dimension carries values two orders of magnitude larger than the rest.

This is especially problematic under max-based per-token scaling. For a signed activation tensor, a common choice is

$$s_x = \frac{\|x\|_\infty}{q_{\max}}, \tag{13}$$

which ensures that the largest feature maps to the largest available code. If one channel has magnitude M and the remaining channels are of order $\varepsilon \ll M$, then most quantization levels are allocated to that single channel. In low-bit settings, many of the smaller features are then quantized to zero or to a very narrow code band. In an ADC-constrained pipeline, this not only harms the input code density, but also leads to poorly occupied output bins after analog accumulation.

A second challenge is the sensitivity of the residual stream. Residual connections propagate local errors across many layers, so local reconstruction loss is not always predictive of end-to-end perplexity. QuaRot explicitly motivates its method by the need to remove outliers from the residual stream [2], and SpinQuant shows that choosing or learning a suitable rotation can dramatically change the accuracy of low-bit LLM inference [19]. QEP later argues that one of the central failures of standard layer-wise PTQ is the accumulation of quantization errors across layers, and proposes explicit quantization-error propagation to mitigate this effect [1]. CBQ reaches a related conclusion from a cross-block reconstruction perspective [9].

A third challenge is architectural non-uniformity. Not all transformer projections are equally sensitive. The output projections `o_proj` and `down_proj` inject their errors directly into the residual stream, whereas earlier projections such as `q_proj`, `k_proj`, `v_proj`, `up_proj`, and `gate_proj` are followed by additional mixing or nonlinear structure before their effect

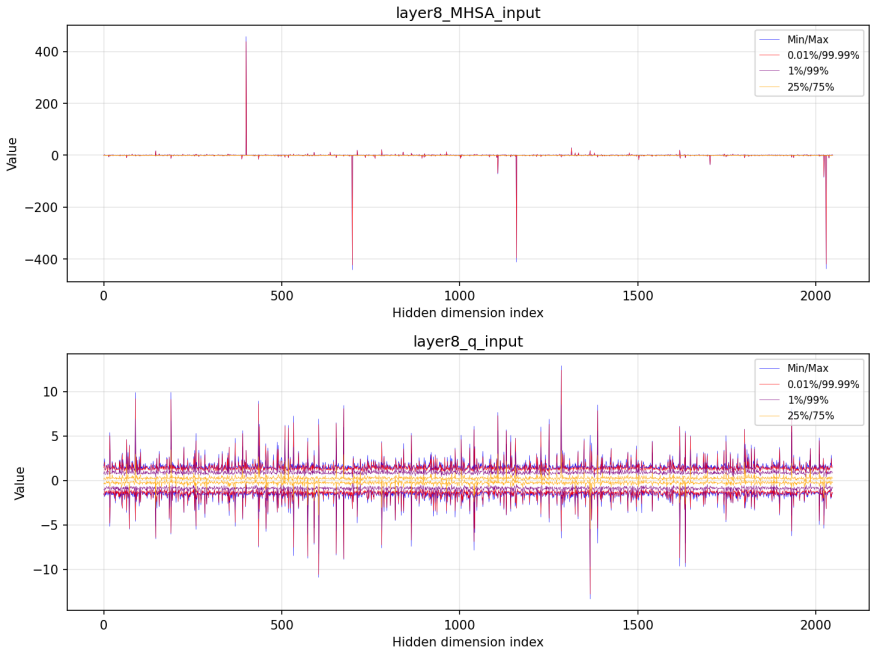

Figure 3: Per-channel activation statistics from a single Llama-3.2-1B decoder block (layer 8), measured over the WikiText-2 calibration set. *Top*: the input to the multi-head self-attention sub-layer (`MHSA input`, i.e. the post-RMSNorm hidden state). A single hidden dimension reaches values on the order of ± 400, while the bulk of channels stays within ± 10. *Bottom*: the input to the query projection (`q_input`) shows the same channel structure compressed into a ± 10 range. The colored bands report min/max, $0.01/99.99\,\%$, $1/99\,\%$, and $25/75\,\%$ percentiles across the calibration tokens. Under max-based per-token scaling (Eq. (13)), a single such outlier channel dominates the scale and forces typical activations into a narrow range of integer codes, which is the underlying cause of dead-zone behaviour in the downstream ADC.

reaches the next block. This asymmetry is one reason why output-side projections often emerge as the most sensitive modules in low-bit transformer inference.

Finally, preserving hidden-state geometry is critical. A transform that reduces scalar range at the price of severely distorting the hidden-state subspace may still harm model quality. Rotation- and affine-transform methods are therefore attractive because they attempt to make the model more quantizable while preserving the underlying full-precision linear map. This point is especially relevant in the present thesis, where the goal is to shape the signal path so that it tolerates both ordinary low-bit quantization and ADC quantization at the output of each analog MVM, beyond simply reducing local error.

2.4 Existing Methods

Research on low-bit PTQ for LLMs has converged around a small number of recurring ideas described in what follows. Two of these methods, FlatQuant and RAOQ, are central to this thesis and receive extended treatment.

The earliest line of work reshapes the distribution of activations before quantisation. SmoothQuant [31] introduces the diagonal equivalent transformation, which absorbs a per-channel diagonal D into the weights so that the linear map is preserved:

$$y \;=\; (xD^{-1})\,(DW)^{\top} \;=\; xW^{\top}. \tag{14}$$

With D chosen from activation and weight statistics, the rescaled activation $\tilde{x} = xD^{-1}$ has reduced peak magnitude, so per-token max-based scaling no longer collapses around a single outlier channel. Outlier Suppression+ [28] extends this scheme with a per-channel shift that handles asymmetric outliers; AWQ [16] uses activation statistics to identify and protect a small set of salient weights; and OmniQuant [25] makes both the diagonal transformation (LET) and a weight-clipping range (LWC) end-to-end learnable, pushing PTQ to very low-bit settings such as W4A4.

These methods reshape the activation distribution but leave the calibration objective itself unchanged. A separate line of work targets that objective directly, treating PTQ as a calibration problem in which the quantisation parameters are fit by minimising reconstruction error against a full-precision reference. ZeroQuant [33] pairs layer-wise distillation with hardware-oriented deployment. CBQ [9] generalises the single-layer objective to cross-block reconstruction, exposing the optimiser to several consecutive transformer blocks at once. QEP [1] feeds quantised hidden states forward during calibration, so that the optimisation target reflects accumulated quantisation error rather than clean teacher activations.

The next line of methods replaces the diagonal scaling of SmoothQuant with a denser linear transformation. QuaRot [2] inserts a fixed Hadamard rotation into the residual stream, removing large-magnitude directions before quantisation. SpinQuant [19] learns this rotation rather than drawing it at random, showing that the choice of rotation substantially affects low-bit accuracy. Both methods act at the level of the residual stream rather than at the level of individual projections.

FlatQuant [18], the calibration backbone of this thesis, sharpens this idea by applying a separate invertible linear transform to each linear projection of the model:

$$y \;=\; x\,W^{\top} \;=\; (x\,T^{-1})\,(T\,W^{\top}) \;=\; \tilde{x}\,\tilde{W}^{\top}. \tag{15}$$

Unlike QuaRot and SpinQuant, the transform T is learned end-to-end through block-level reconstruction, so every projection has its own tailored transform that flattens the local activation distribution. To keep the per-projection parameter count tractable, the $d \times d$ transform is parameterised as a Kronecker product

$$T \;=\; T_1 \otimes T_2, \qquad T_1 \in \mathbb{R}^{a\times a}, \quad T_2 \in \mathbb{R}^{b\times b}, \quad ab = d, \tag{16}$$

reducing the parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$. After calibration, T is folded into the surrounding weights, so the inference path is unchanged. At W4A4, FlatQuant set the strongest published PTQ result on Llama-3 at the time of writing.

The methods discussed so far operate entirely in the digital domain. RAOQ [27] is the first to make the analog-to-digital boundary itself an object of optimisation. Unlike the PTQ methods above, RAOQ is a quantisation-aware training (QAT) method: it starts from a pre-trained model and fine-tunes weights and activations end-to-end with the ADC included in the forward pass, targeting the variance of the pre-ADC accumulation $\mathrm{Var}[Y]$, which sets the signal-to-quantisation-noise ratio of the analog readout. Activation shifting (A-shift) re-quantises signed activations as unsigned and shifts them by a constant $C = 2^{b_x-1}$,

$$x_s \;=\; \mathrm{clip}\!\left(\left\lfloor \frac{x-\beta}{s_x} \right\rceil, 0,\, 2^{b_x}-1\right) - C, \tag{17}$$

which concentrates the mass of x_s at large absolute values and increases $\mathbb{E}[X^2]$ by roughly $15\times$ compared with direct signed quantisation; the resulting offset $C\sum_i w_i$ in the IMC computation is precomputed offline and adds no inference overhead. Weight reshaping (W-reshape) adds a kurtosis penalty to the QAT loss to flatten the weight distribution and increase $\mathrm{Var}[W]$:

$$\kappa \;=\; \mathbb{E}\left[\left(\frac{W-\mu_W}{\sigma_W}\right)^4\right], \qquad L_Q \;=\; L_c + \lambda_\kappa \sum_\ell \kappa_\ell, \tag{18}$$

with μ_W, σ_W the mean and standard deviation of the weight tensor and L_c the task loss. Bit augmentation (BitAug) trains the model under multiple ADC bit precisions simultaneously,

$$L_A \;=\; L(\theta, b_a) + \lambda_b\, L(\theta, \tilde{b}_a), \tag{19}$$

with $\tilde{b}_a$ sampled at each iteration from a neighbourhood of the target bit width; the extra gradients smooth the ADC-induced loss surface and reduce the chance of getting stuck in local minima. For models with more than $\sim$200M parameters, RAOQ also fits a LoRA-style adapter (ADC-LoRA) inside the quantised weight, $W + AB$, to reduce the gradient-accumulation cost of BitAug; because AB is summed into the weight before the ADC, this is a pre-ADC LoRA. Analog Foundation Models [4] take a different position within the same QAT-style family: rather than tuning a small set of parameters, the entire LLM is fine-tuned under explicit analog noise constraints, trading training cost for stronger end-to-end accuracy.

When pure PTQ saturates, a natural next step is to freeze the quantised backbone and attach a small trainable correction. LLM-QAT [17] applies quantisation-aware training to extremely low-bit transformers, and EfficientQAT [6] reduces the cost of this idea through block-wise parameter updates and end-to-end training of the quantisation parameters. A related sub-line treats the correction as low-rank: INT2.1 [5], LQER [34], Low-Rank Correction for Quantized LLMs [24], and LR-QAT [3] all show that much of the residual quantisation error can be captured by training a small low-rank addition to each linear layer.

2.5 Research Gap

The literature reviewed above establishes two complementary facts. First, state-of-the-art PTQ methods for LLMs (scaling- and rotation-based approaches, as well as reconstruction-based ones) are highly effective at reducing input-side activation and weight quantization error [31, 16, 25, 2, 19, 18]. Second, IMC-oriented work shows that analog output quantization introduces a separate and often dominant bottleneck that is not naturally captured by ordinary digital PTQ objectives [27, 4].

This creates a specific gap for ADC-constrained LLM inference. Standard PTQ methods typically optimize a local reconstruction objective

$$\mathcal{L}_{\mathrm{PTQ}} = \sum_{x \in \mathcal{C}} \left\| f_\ell(x) - \hat{f}_\ell(x) \right\|_2^2, \tag{20}$$

possibly with clipping or affine preconditioning. Even when quantization-error propagation is included, the target is usually still a digital hidden-state reconstruction objective [1]. In an IMC accelerator, however, the effective layer map is

$$\hat{f}_\ell^{\mathrm{ADC}}(x) = D_{\mathrm{out}}\, Q_{\mathrm{ADC}}(Q_w(W_\ell)\, Q_x(x))\,, \tag{21}$$

so the model is constrained not only by low-bit inputs and weights, but also by the ADC quantization of the analog accumulation. Consequently, a method can achieve strong ordinary PTQ metrics and still perform poorly once the ADC is inserted into the inference loop.

ADC-aware QAT methods such as RAOQ [27] and Analog Foundation Models [4] have shown that this output-side bottleneck can in principle be closed by fine-tuning the entire network end-to-end with the ADC in the loop. The cost of this approach scales poorly with model size: full QAT on a billion-parameter LLM requires hours to days of GPU time per configuration, full-precision gradients through every quantised layer, and additional memory for techniques like BitAug that accumulate gradients across several ADC bit widths. ADC-aware QAT has therefore been demonstrated mostly on convolutional models and on language models in the 100M–1B range, and does not yet scale comfortably to the contemporary LLM regime. The first question raised by this gap is whether transform-based PTQ, which is much cheaper than full QAT, can be extended to be ADC-aware while retaining model quality.

A second gap appears after pure PTQ saturates. Once the best transform-based ADC-aware PTQ solution has been found, the remaining error may no longer be well addressed by additional clipping, scaling, or rotation heuristics. At that point, the relevant question is whether the frozen ADC path can be corrected by a lightweight residual branch rather than by retraining the full model. Although low-rank correction has been studied in digital low-bit settings [34, 24, 3], its interpretation as a post-ADC residual correction on top of a frozen IMC signal path is not standard in the LLM PTQ literature.

The research gap of this thesis is therefore twofold:

1. existing PTQ methods mainly optimize input-side quantization error, while ADC-constrained inference introduces a distinct output quantization bottleneck that must be modeled explicitly;
2. once transform-based PTQ saturates, a lightweight residual correction stage may be required, but it is unclear where such a correction should be inserted and how it should be trained.

The following chapters address these gaps in two stages. First, they extend transform-based PTQ with ADC-aware objectives, error propagation, and bounded equivalent scaling. Second, they investigate low-rank residual correction on top of the frozen quantized ADC path, showing that post-ADC correction can recover a large fraction of the remaining accuracy loss in low-bit LLaMA inference.

3 Problem Statement and Research Questions

3.1 Problem Definition

The core problem studied in this thesis is how to deploy a transformer LLM on an in-memory computing (IMC) inference path in which every large linear projection is constrained both by low-bit weight and activation quantization and by a finite-resolution analog-to-digital converter (ADC) after the matrix–vector multiplication. In a standard digital setting, a linear projection in a transformer block computes

$$y = xW^\top + b, \tag{22}$$

where $x \in \mathbb{R}^{d_{\text{in}}}$ is the input hidden state, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the projection matrix, and b is an optional bias term. In the IMC setting considered here, the same operation is executed tile by tile in quantized code space.

Let the input dimension be split into tiles of width M. For tile t, the activation and weight tensors are quantized as

$$x_t \approx s_{x,t}\hat{x}_t, \qquad W_t \approx \operatorname{diag}(\mathbf{s}_{w,t})\hat{W}_t, \tag{23}$$

where $\hat{x}_t$ and $\hat{W}_t$ are integer activation and weight codes, $s_{x,t}$ is the activation scale, and $\mathbf{s}_{w,t}$ is the vector of per-output-channel weight scales. The analog array accumulates the integer dot product

$$z_t = \hat{x}_t \hat{W}_t^\top. \tag{24}$$

Unlike conventional digital quantization, this accumulated value is not immediately available at full integer precision. It must first be discretized by the ADC:

$$Q_{\text{ADC}}(z_t; \Delta, n_a, p_a) = \Delta \cdot \operatorname{clip}\left(\left\lfloor \frac{z_t}{\Delta} \right\rfloor, n_a, p_a\right), \tag{25}$$

where Δ is the ADC step size and (n_a, p_a) are the lower and upper ADC code bounds. The dequantized output of the tiled projection is then

$$\hat{y} = b + \sum_t s_{x,t}\left(\mathbf{s}_{w,t} \odot Q_{\text{ADC}}(z_t; \Delta, n_a, p_a)\right), \tag{26}$$

where $\odot$ denotes element-wise multiplication.

The ADC step size is a hardware-defined quantity. For the signed bipolar ADC model used in the main experiments, it is given by

$$\Delta = \frac{2Mq_xq_w}{2^{b_a}k}, \tag{27}$$

where b_a is the ADC bit-width, k is the ADC scaling factor, and q_x, q_w are the maximum activation and weight code levels determined by the activation and weight precisions b_x

and b_w. In the unsigned shift-subtract setting, the same role is played by the unipolar step size

$$\Delta_u = \frac{M(2^{b_x}-1)(2^{b_w}-1)}{(2^{b_a}-1)\,k}. \tag{28}$$

In both cases, the essential property is the same: the output lattice is fixed by hardware parameters and cannot be made finer by ordinary post-training calibration.

This makes ADC-constrained inference fundamentally different from standard digital post-training quantization (PTQ). A PTQ method may successfully reduce the error in $\hat{x}_t$ and $\hat{W}_t$, while the accumulated output z_t may still be mapped to a coarse ADC lattice. If $|z_t| < \Delta$, the output falls into the ADC dead zone and may be mapped to zero. If z_t exceeds the ADC range, the output is clipped. Therefore, the objective is to quantize weights and activations accurately and to shape the internal integer accumulation so that the ADC range is used effectively.

The software-side degrees of freedom considered in this thesis are transform-based PTQ parameters and, in the second stage, a lightweight residual adaptation branch. Following the FlatQuant idea, an invertible transform T_ℓ can be inserted into a linear projection without changing the full-precision computation:

$$xW_\ell^\top = (xT_\ell^{-1})(T_\ell W_\ell^\top). \tag{29}$$

However, after quantization and ADC discretization, this equivalence no longer holds exactly. The practical calibration problem is therefore to choose transforms that minimize the mismatch between the full-precision block output and the ADC-constrained quantized block output:

$$\min_{\phi}\ \mathbb{E}_{x\in\mathcal{C}}\left[\left\|B_\ell(x) - \hat{B}_\ell^{\mathrm{ADC}}(x;\phi,\mathcal{H})\right\|_2^2\right], \tag{30}$$

where B_ℓ is the full-precision transformer block, $\hat{B}_\ell^{\mathrm{ADC}}$ is its quantized ADC-constrained counterpart, ϕ denotes the PTQ calibration parameters, $\mathcal{C}$ is the calibration set, and

$$\mathcal{H} = (b_x, b_w, b_a, M, k, \mathrm{ADC\ mode}) \tag{31}$$

is the fixed hardware configuration.

At the model level, the goal is to obtain an ADC-constrained model $\hat{F}^{\mathrm{ADC}}$ whose quality remains close to the full-precision model F and to the corresponding digital quantized model with the ADC disabled:

$$\min_{\phi,\psi}\ \mathrm{PPL}\left(\hat{F}^{\mathrm{ADC}}_{\phi,\psi};\mathcal{D}_{\mathrm{eval}}\right), \qquad \text{subject to fixed } \mathcal{H} \text{ and } |\psi| \ll |\theta|. \tag{32}$$

Here $\psi = 0$ corresponds to pure PTQ, while ψ denotes the optional lightweight residual correction parameters used in the adaptation stage. The central question is whether ADC-aware PTQ alone is sufficient, or whether the fixed ADC output lattice creates a residual error that must be corrected by a small trainable module.

3.2 Research Questions

This thesis is guided by the following research questions.

RQ1. How much quality degradation is caused specifically by ADC output quantization in LLMs?

This question separates ordinary digital quantization error from the additional degradation introduced by the ADC. The comparison is made between full-precision inference, quantized inference with the ADC disabled, and quantized inference with the full ADC path enabled. The goal is to determine whether the ADC is a minor implementation detail or a dominant source of error under low-bit IMC inference.

RQ2. To what extent can FlatQuant-style post-training transformations mitigate ADC-induced degradation?

This question studies whether learned equivalent transformations can reshape activation and weight distributions so that the integer accumulation uses the ADC range more effectively. In particular, the thesis evaluates block-wise calibration, propagated quantized inputs, mixed full-precision/ADC objectives, trainable diagonal scaling, and staged training.

RQ3. Is reconstruction-only PTQ sufficient under ADC constraints, or are explicitly ADC-aware objectives required?

Standard PTQ methods usually minimize reconstruction error under digital quantization assumptions. This thesis investigates whether such objectives remain predictive once the ADC is inserted into the inference path. The question is answered by comparing clean-input reconstruction, ADC-propagated reconstruction, and objectives that explicitly expose the calibration process to ADC noise.

RQ4. Which layers and projection types are most sensitive to ADC quantization?

LLaMA decoder blocks contain several linear projections: q, k, v, o, gate, up, and down. These projections do not contribute equally to end-to-end degradation. This question identifies whether ADC sensitivity is concentrated in particular projections, such as output-side attention and MLP projections, or distributed uniformly across the architecture.

RQ5. Under what conditions does ADC-constrained inference require lightweight model adaptation beyond PTQ?

If transform-based PTQ reaches a quality plateau, the remaining error may be caused by the finite ADC output resolution rather than by poorly calibrated weights or activations. This question evaluates whether a small post-ADC residual correction branch can close the remaining quality gap while keeping the quantized ADC backbone frozen.

Together, these questions connect three levels of analysis: the hardware-level ADC discretization, the layer-level behavior of quantized transformer projections, and the model-level language modeling quality.

3.3 Scope and Assumptions

The thesis focuses on a decoder-only transformer LLM, with Llama-3.2-1B used as the main experimental target. This model is large enough to exhibit the activation outlier and residual-stream sensitivity typical of modern LLMs, while still being small enough to

support extensive ablation studies over quantization precision, ADC settings, calibration strategies, and residual adaptation variants.

The hardware behavior is studied through simulation rather than through measurements on a fabricated IMC chip. The simulator models tiled integer matrix–vector multiplication, per-token activation quantization, per-channel weight quantization, ADC floor quantization, ADC clipping, and dequantization. Two ADC settings are considered: a signed bipolar ADC and an unsigned unipolar shift-subtract ADC. The hardware parameters M, b_x, b_w, b_a, and k are treated as fixed during each experiment unless an explicit per-layer k-search experiment is performed.

Several hardware effects are outside the scope of this work. The simulator does not model device-to-device variation, thermal noise, conductance drift, IR drop, nonlinear crossbar behavior, write noise, peripheral circuit latency, or full chip-level energy consumption. The purpose of the study is therefore not to claim final hardware efficiency for a specific IMC implementation, but to isolate the algorithmic consequences of ADC output discretization.

The quantization study is restricted to the main linear projections inside the transformer decoder blocks. Other components, such as layer normalization, nonlinearities, softmax, rotary embeddings, and token embeddings, are not the primary target of the ADC simulation. This restriction reflects the fact that large linear projections dominate transformer inference cost and are the most natural mapping target for IMC arrays.

The main methodological focus is post-training quantization. The thesis does not retrain the full model and does not modify the original pretraining objective. When adaptation is considered, it is limited to a small residual low-rank branch trained on top of a frozen quantized ADC path. This allows the thesis to distinguish between what can be achieved by calibration alone and what requires additional trainable parameters.

The calibration data is finite and substantially smaller than the original pretraining data. Therefore, the reported results should be interpreted as evidence about practical calibration and adaptation under limited data, not as an upper bound on what could be achieved with large-scale quantization-aware training.

3.4 Evaluation Criteria

The primary model-level metric is perplexity. For each configuration, the thesis compares

$$\mathrm{PPL}_{\mathrm{FP}}, \qquad \mathrm{PPL}_{\mathrm{no\,ADC}}, \qquad \mathrm{PPL}_{\mathrm{ADC}}, \tag{33}$$

where $\mathrm{PPL}_{\mathrm{FP}}$ is the full-precision baseline, $\mathrm{PPL}_{\mathrm{no\,ADC}}$ measures quantized inference with the ADC disabled, and $\mathrm{PPL}_{\mathrm{ADC}}$ measures the full ADC-constrained inference path. The difference between no-ADC and ADC perplexity isolates the degradation introduced by output quantization:

$$\Delta_{\mathrm{ADC}} = \mathrm{PPL}_{\mathrm{ADC}} - \mathrm{PPL}_{\mathrm{no\,ADC}}, \tag{34}$$

while the ratio

$$\rho_{\mathrm{ADC}} = \frac{\mathrm{PPL}_{\mathrm{ADC}}}{\mathrm{PPL}_{\mathrm{no\,ADC}}} \tag{35}$$

measures the relative severity of the ADC bottleneck. A successful ADC-aware method should reduce both $\mathrm{PPL}_{\mathrm{ADC}}$ and the no-ADC-to-ADC gap without severely degrading no-ADC quality.

In addition to perplexity, the thesis uses internal ADC-level metrics to diagnose why a configuration succeeds or fails. The dead-zone rate is defined as the fraction of pre-ADC accumulated values whose post-ADC code is exactly zero,

$$r_{\text{dead}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1} \left[\lfloor z_i/\Delta \rfloor = 0 \right], \tag{36}$$

where z_i denotes a pre-ADC accumulated value. This is the floor-consistent definition: $\lfloor z_i/\Delta \rfloor = 0$ holds for $z_i \in [0, \Delta)$ in the bipolar ADC and for $z_i \in [0, \Delta)$ in the unipolar ADC. A high dead-zone rate indicates that many analog outputs are too small to register on the ADC lattice. The clipping rate is defined as

$$r_{\text{clip}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1} \left[c_i = n_a \ \text{ or } \ c_i = p_a \right], \tag{37}$$

where c_i is the ADC code after discretization and clipping. This metric detects whether the integer accumulation systematically exceeds the representable ADC range.

The thesis also evaluates ADC bin utilization:

$$u_{\text{bin}} = \frac{|\{c_i : i = 1, \ldots, N\}|}{2^{b_a}}, \tag{38}$$

which measures how much of the available ADC codebook is actually used. Low bin utilization indicates that the ADC behaves like a much lower-precision device than its nominal bit-width suggests.

Layer-level reconstruction quality is measured by the relative error

$$e_\ell = \frac{\|\hat{y}_\ell - y_\ell^\star\|_2^2}{\|y_\ell^\star\|_2^2}, \tag{39}$$

where $y_\ell^\star$ is the full-precision reference output and $\hat{y}_\ell$ is the quantized ADC output. This metric is used to identify local failures even when the final perplexity alone does not reveal their source.

Finally, architectural sensitivity is evaluated by comparing the behavior of different projection types and layer groups. A projection is considered ADC-sensitive if improving, bypassing, or correcting it produces a disproportionate reduction in end-to-end perplexity or reconstruction error. This criterion is especially important for distinguishing projections that primarily affect local reconstruction from projections whose errors are injected directly into the residual stream.

For residual adaptation experiments, success is evaluated by perplexity reduction together with parameter efficiency and placement. A useful adaptation method should keep the ADC backbone frozen, add only a small fraction of new parameters, train stably, generalize beyond the calibration distribution, and outperform pre-ADC correction baselines. These criteria ensure that the adaptation stage remains a lightweight correction to the ADC path rather than a full quantization-aware retraining procedure.

4 Proposed Method

4.1 Post-Training Quantization

4.1.1 Baseline: FlatQuant for LLaMA

FlatQuant [18] exploits the linear invariance of the matrix-vector product to redistribute the dynamic range of activations before quantization. For any invertible transform $T \in \mathbb{R}^{d\times d}$,

$$y = xW^\top = \underbrace{(xT^{-1})}_{\tilde{x}}\,\underbrace{(TW^\top)}_{\tilde{W}^\top}.$$

Choosing T to flatten the distribution of $\tilde{x}$ improves both integer quantization accuracy and ADC bin coverage. Computing a full $d \times d$ transform is prohibitive, so T is Kronecker-factorised as $T = T_1 \otimes T_2$ with $T_1 \in \mathbb{R}^{a\times a}$ and $T_2 \in \mathbb{R}^{b\times b}$ for $ab = d$, reducing the parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$. For Llama-3.2-1B ($d = 2048$), $a = 32$ and $b = 64$ are used.

Calibration proceeds layer by layer in a teacher-student fashion. For each linear projection, a frozen floating-point (FP) teacher produces reference outputs $y^\star = L_{\mathrm{FP}}(x^\star)$, and the quantized student $\hat{y} = \hat{L}_T(\hat{x})$ minimizes the mean-squared error $\|\hat{y} - y^\star\|^2$ over the Kronecker factors only, with weights held at their quantized values. After calibration, T is folded into the weights via reparameterisation; inference cost equals that of standard integer quantization.

FlatQuant is adapted to the LLaMA-3 architecture [10] by wrapping all seven linear projections in each decoder block: q, k, v, o in the grouped-query attention sub-layer, and `gate`, `up`, `down` in the SwiGLU feed-forward network.

4.1.2 ADC Hardware Model and Forward Simulation

In an analog in-memory compute (AIMC) accelerator, quantized weights $\hat{w} \in [-q_w, q_w]^M$ are stored on a crossbar tile of size M. At inference, a quantized activation tile $\hat{x} \in [-q_x, q_x]^M$ is presented, the crossbar accumulates their integer dot product $y_{\mathrm{int}} = \langle \hat{w}, \hat{x} \rangle$ in the analog domain, and an ADC floor-quantizes the result with a fixed step size δ:

$$\hat{y} \;=\; \delta \cdot \mathrm{clip}\Big(\Big\lfloor \frac{y_{\mathrm{int}}}{\delta} \Big\rfloor ,\; -2^{b_a-1},\; 2^{b_a-1}-1 \Big).$$

The step size is a hardware constant set at manufacturing time:

$$\delta = \frac{2\,M\,q_x\,q_w}{2^{b_a}\,k},$$

where b_a is the ADC bit-width and k is the number of ADC channels operated in parallel. Two failure modes follow directly. The dead zone is asymmetric under floor quantization: values in $0 \le y_{\mathrm{int}} < \delta$ map to the zero code, whereas small negative values $-\delta \le y_{\mathrm{int}} < 0$ map to the -1 code rather than zero. The dead-rate is therefore measured directly as the fraction of post-ADC codes equal to zero (Eq. 36). Clipping: if $|y_{\mathrm{int}}| > \delta \cdot (2^{b_a-1}-1)$, the output saturates. For the target configuration ($b_w = b_x = 4$, $M = 256$, $b_a = 8$, $k = 16$) this gives $\delta \approx 6.12$, coarse enough that a naïve INT4 baseline shows a no-ADC-to-ADC perplexity gap of $\times$153 (15.41 vs. 2354).

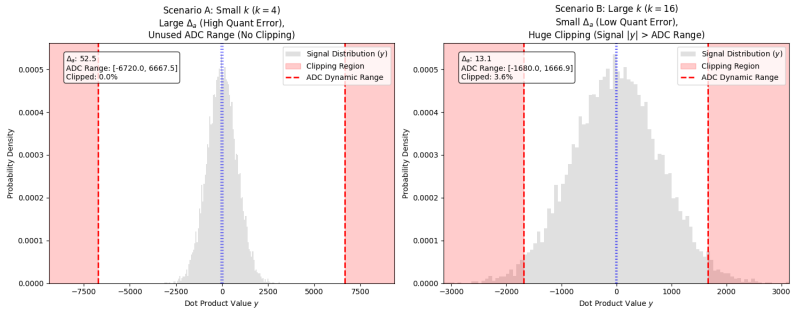

Figure 4: ADC dead-zone illustration. Pre-ADC values in $[0, \delta)$ floor to the zero code, discarding the analog signal entirely. The two failure modes (dead zone and clipping) are determined by the fixed step δ and the b_a-bit ADC range.

4.1.3 Limitations of Per-Layer Reconstruction

Standard FlatQuant minimizes per-projection MSE feeding each layer its clean FP activations from the calibration set. Two gaps make this insufficient under ADC inference. First, the calibration objective is blind to the ADC step: a projection optimized in isolation may produce pre-ADC signals that fall inside the dead zone after re-quantization of the next tile. Second, error accumulates across layers at inference but not during calibration; the input to layer ℓ at inference is the noisy, ADC-distorted output of layer $\ell-1$, not the clean FP tensor the optimizer saw. These gaps motivate all three ADC-aware extensions described below.

4.1.4 Block-wise Calibration with Propagation and α-Mixing

Block-level reconstruction. Rather than minimising per-projection MSE independently, the output of each entire transformer block is reconstructed:

$$\mathcal{L} = \left\| \hat{B}(\hat{x}) - B_{\mathrm{FP}}(x^{\star}) \right\|_2^2.$$

Gradients propagate through the full block ($q/k/v \to \mathrm{attn} \to o \to \mathrm{gate/up} \to \mathrm{down}$), so the transforms of earlier projections are aware of how their error propagates to later ones within the same block.

Propagation across blocks. Blocks are calibrated sequentially from layer 0 to layer $N-1$. When training layer ℓ, its input is the ADC-quantized output produced by the already-trained layer $\ell-1$, not a clean FP activation. Error thus accumulates during calibration in the same way it does at inference, closing the distribution mismatch between training and deployment that per-layer calibration leaves open.

α-mixed objective. A purely ADC-propagated loss produces noisy gradients because the straight-through estimator must pass through multiple floor operations. The FP and ADC losses are interpolated with a scalar $\alpha \in [0, 1]$:

$$\mathcal{L}(\alpha) = (1 - \alpha)\, \mathcal{L}_{\mathrm{FP}} + \alpha\, \mathcal{L}_{\mathrm{ADC}}, \tag{40}$$

where $\mathcal{L}_{\mathrm{FP}}$ uses the clean FP input and $\mathcal{L}_{\mathrm{ADC}}$ uses the quantized input from the previous block. $\alpha = 0$ recovers clean calibration; $\alpha = 1$ is full propagation. In the INT8 setting, full

propagation ($\alpha = 1$) alone drops ADC perplexity from 28.86 to 14.55, and adding a small bin-center regularizer further reduces it to 14.41; an α-sweep is unnecessary because the noise from full propagation is already manageable. In the INT4 setting full propagation is too noisy: an α-sweep (Section 5) shows that $\alpha = 0.5$ gives the best trade-off: below 0.5 the loss under-weights the ADC regime, while above 0.5 gradient noise from the heavier quantization degrades transform quality.

4.1.5 Trainable Per-Channel Diagonals

A formal bound is first established to show that orthogonal Kronecker transforms cannot suppress per-channel outliers beyond a fixed threshold. This motivates augmenting the transform with an anisotropic component: a per-channel diagonal factor is the simplest such augmentation that preserves the weight-activation invariance and supplies the missing degree of freedom, although other parameterizations could in principle achieve a similar effect.

Lemma 4.1 (Rotation-only outlier suppression bound)**.** *Let $P = L \otimes R$ with $L^\top L = I$, $R^\top R = I$ be an orthogonal Kronecker transform on $\mathbb{R}^d$ ($d = a \cdot b$, $L \in \mathbb{R}^{a\times a}$, $R \in \mathbb{R}^{b\times b}$). For any $x \in \mathbb{R}^d$ define the effective participation ratio*

$$m_{\text{eff}}(x) \;=\; \frac{\|x\|_2^2}{\|x\|_\infty^2} \;\in\; [1,\, d].$$

Then for every orthogonal Kronecker transform P,

$$\|xP\|_\infty \;\geq\; \frac{\|x\|_2}{\sqrt{d}},$$

and consequently the maximum achievable suppression of the peak activation over all such P satisfies

$$\frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \;\leq\; \sqrt{\frac{d}{m_{\text{eff}}(x)}}.$$

Proof. Since P is orthogonal, $\|xP\|_2 = \|x\|_2$. For any vector $y \in \mathbb{R}^d$ the standard ℓ^∞–ℓ^2 inequality gives $\|y\|_\infty \geq \|y\|_2/\sqrt{d}$. Applying this to $y = xP$ yields the first inequality. The suppression ratio bound then follows by dividing $\|x\|_\infty = \|x\|_2/\sqrt{m_{\text{eff}}(x)}$ by the lower bound $\|xP\|_\infty \geq \|x\|_2/\sqrt{d}$. □

Corollary 4.2 (Kronecker capacity)**.** *A full orthogonal matrix on $\mathbb{R}^d$ has $d(d-1)/2$ free parameters. The Kronecker family $L \otimes R$ has only $a(a-1)/2 + b(b-1)/2$ free parameters; for $d = 2048 = 32 \times 64$ this is 2512 vs. $\approx 2.1 \times 10^6$. Consequently, the Kronecker family is a strict low-dimensional subset of all orthogonal transforms and may not attain the bound in Lemma 4.1; the true achievable suppression ratio can be strictly smaller.*

Implications for INT4 ADC. In the present setting, per-token activation scaling sets $s_x = \|x\|_\infty / q_{\max}$. If a small number of channels persistently dominate (i.e. $m_{\text{eff}} \ll d$), Lemma 4.1 gives a ceiling on how much rotation alone can help. For example, with $d = 2048$ and 8 dominant channels ($m_{\text{eff}} \approx 8$), the maximum suppression factor is $\sqrt{2048/8} = 16$; if the required suppression exceeds this, rotation-only calibration is provably insufficient. The INT4 baseline confirms this empirically: even after FlatQuant rotation, the no-ADC-to-ADC perplexity gap remains at $\times 153$ (15.41 vs. 2354.86), and the

dead-rate stays at $\sim 10\%$. Adding diagonal scaling collapses the gap to $\times 1.4$ at the same dead-rate, showing that the residual ADC error after rotation is driven by coarse output quantization rather than by dead zones.

Motivation and formulation. A Kronecker rotation reshapes the covariance of an activation cloud but does not adjust per-channel scales. Under INT4, residual outlier channels remain after rotation: a few channels with magnitudes 5–$10\times$ the median force the per-token scale factor to waste range on those channels and under-represent the rest, which directly worsens the ADC dead-zone rate because y_{int} per output channel depends on how uniformly codes are distributed.

A trainable diagonal $D = \text{diag}(d_1, \ldots, d_c)$, $d_i \in [10^{-4}, 10]$, is absorbed into the rotation, $\tilde{T} = T \cdot D$. The linear map is preserved by applying equal-and-opposite rescaling to the weights:

$$y = \underbrace{(x\, D^{-1} T^{-1})}_{\tilde{x}} \underbrace{(T\, D\, W^{\top})}_{\tilde{W}^{\top}}.$$

The bounds on d_i prevent degenerate scaling. D is trained jointly with T during block-level calibration and folded into $\tilde{W}$ at deployment, adding no inference overhead. Crucially, because D is not orthogonal, it breaks the ℓ^2-energy invariance assumed in Lemma 4.1: picking $d_j > 1$ amplifies row j of $\tilde{W}$ (where it is absorbed cleanly by per-channel weight scaling) and equivalently shrinks channel j of the rotated activation $\tilde{x}$ by a factor $1/d_j$. For an outlier activation channel this is exactly the per-channel rebalancing that per-token activation scaling cannot perform.

Placement in attention and MLP blocks. Separate diagonal matrices are placed before the attention sub-layer (`diag_attn`, shared by $q/k/v$, plus a second instance before o) and before the FFN sub-layer (`diag_mlp`, shared by `gate`/`up`, plus a second before `down`). An ablation (Section 5) shows that both placements are necessary: `diag_mlp` drives the no-ADC perplexity improvement while `diag_attn` closes the ADC gap; either alone leaves significant perplexity on the table.

4.1.6 Staged Training

Training all Kronecker factors and diagonals jointly from scratch with $\alpha = 0.5$ is unstable: attention gradients dominate early optimization before MLP transforms have converged, yielding a poor initialization for both sub-systems.

Calibration is split into two stages. Stage A (30 epochs, learning rate η) trains only the MLP Kronecker factors $T_{\text{gate}}, T_{\text{up}}, T_{\text{down}}$ and MLP diagonals $D^{(\text{m})}$ with $\alpha = 0$ (clean FP inputs). The FFN transforms therefore converge in a stable, noise-free regime. Stage B (10 epochs, learning rate 0.1η) unfreezes the attention Kronecker factors T_q, T_k, T_v, T_o and attention diagonals $D^{(\text{a})}$, and switches to $\alpha = 0.5$. The MLP warm start prevents gradient interference from attention. Among configurations that reach the pure-PTQ INT4 plateau (ADC perplexity ≈ 27.6), this procedure attains the best no-ADC perplexity (18.50), indicating a better-conditioned underlying transform than the joint Diag + α=0.5 baseline at ADC PPL 27.56.

4.2 Quantization-Aware Adaptation via Post-ADC Residual LoRA

PTQ methods reach a floor around 27.5–28.5 ADC perplexity for INT4 regardless of calibration strategy. The limiting factor is ADC resolution: with $\delta \approx 6.12$ each output channel has at most $\sim$30 usable discrete levels, introducing irreducible rounding error that no linear transform can eliminate. PTQ is complemented by a lightweight adaptation step that learns to correct ADC distortion after it has occurred, in the digital domain, without modifying the frozen quantized model.

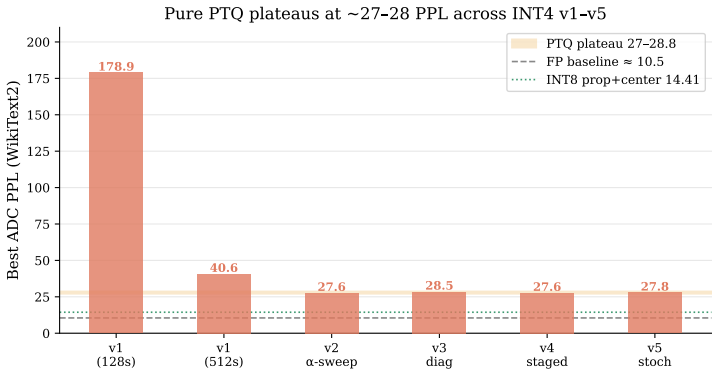

Figure 5: Pure PTQ plateau. After all transform-based PTQ improvements (sample count, α-mixing, diagonals, staged training), INT4 ADC perplexity saturates around 27.5–28.5, independent of calibration strategy. The remaining gap to the FP and INT8 baselines motivates the post-ADC residual correction stage.

4.2.1 Architecture

The PTQ checkpoint is frozen entirely. For each of the seven linear projections in every decoder block, a parallel low-rank branch is added that reads the full-precision input x_{fp} and adds its output to the ADC result:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantized path}} + \underbrace{\frac{\alpha_r}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r} ,$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are LoRA [13] factors stored and computed in `float32`, and α_r/r is the standard LoRA scaling. The values $\alpha_r = 8$ and $r = 4$ are used (scaling factor 2.0). All base model parameters (weights, Kronecker factors, diagonals, and ADC step sizes) remain frozen throughout. Table 1 reports the parameter overhead of rank-4 adapters covering all seven projections across the 16 decoder blocks; the count is small relative to the base model.

Honest positioning of the correction. The base ADC path remains frozen and IMC-compatible: every weight, rotation, and diagonal can still be folded into the analog crossbar. The residual branch is a lightweight *digital* correction: it reads the full-precision input and computes a $\text{float}32$ low-rank matmul *after* the ADC. It therefore adds parameters and digital MACs and is not itself executed on the analog array. A future hardware design would need to decide whether this correction is computed digitally, approximated within

Table 1: Post-ADC LoRA overhead for Llama-3.2-1B ($d_{\text{model}} = 2048$, $d_{\text{ffn}} = 8192$, $d_{\text{kv}} = 512$, 16 layers).

Target	rank	added params	% of model	extra MACs/token
all 7 projections	4	2.82M	0.228%	2.82M
all 7 projections	8	5.64M	0.456%	5.64M
`down` + `o_proj`	4	0.92M	0.074%	0.92M

the IMC tile, or applied selectively only to the most ADC-sensitive projections; in this thesis it is evaluated as a software-only adaptation stage on top of the frozen IMC-compatible base model.

4.2.2 Training Objective

The adapters are trained for 5 epochs with learning rate 10^{-4} using AdamW, on the same 128-sample calibration set used for PTQ. The loss combines cross-entropy on the next-token prediction with a knowledge-distillation term toward the FP teacher:

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p},\, y) + \lambda\, \text{KL}\big(\hat{p}_T \,\big\|\, p_{\text{FP},T}\big)\,,$$

with $\lambda = 0.5$ and temperature $T = 2.0$ on both teacher and student logits. The FP teacher forward pass is computed once per batch with no gradient. Ablations (Section 5) show that the KL term improves ADC perplexity over CE alone, that rank $r = 4$ saturates the gain, and that covering all 16 decoder blocks is necessary because ADC distortion accumulates across depth.

4.2.3 Why Post-ADC Placement

Placing the LoRA correction before the ADC, modifying the pre-ADC integer signal, fails to converge. Any correction $\Delta = BAx$ is itself quantized by the fixed step δ before reaching the output, so it cannot express sub-δ adjustments. Moreover, gradients through the floor operator are too noisy for stable training from a cold start, and the coupled interaction with the hard ADC clamp causes the training run to diverge.

Post-ADC placement avoids both problems: the correction is added in the digital domain after the ADC read, at full floating-point precision, with clean gradients from the CE+KL loss. ADC perplexity drops from the PTQ plateau of ≈ 27.6 to 14.03 for INT4 on WikiText-2, surpassing the INT8 PTQ result (14.41), and the same configuration improves C4 ADC perplexity from the PTQ baseline of 46.40 to 23.30, confirming that the correction generalizes beyond the calibration domain.

5 Experiments and Results

5.1 Model, Data, and Hardware Setup

Model. All experiments use Llama-3.2-1B [10] in `bfloat16`. The model has 16 decoder layers, hidden dimension $d = 2048$, 32 attention heads with grouped-query attention, and a SwiGLU FFN with intermediate dimension 8192. It is small enough to run a full sweep

of dozens of configurations overnight on a single GPU while being representative of the transformer architecture used in larger LLMs.

Calibration data. FlatQuant transforms and LoRA adapters are calibrated on WikiText-2 training text [20]. Unless stated otherwise, 128 randomly sampled sequences of 2048 tokens each are used. Selected experiments use 512 or 1024 samples to study the effect of calibration set size; the INT4 series demonstrates that sample count is a dominant lever for this bit-width.

Evaluation. Perplexity is measured on the WikiText-2 test set using a sliding window of context length 2048 and stride 1024. Selected configurations are also evaluated on C4 [23] under the same protocol to verify out-of-domain generalization.

ADC hardware parameters. Table 2 lists the fixed hardware constants used throughout. All values are chosen to match a plausible near-term AIMC chip target. The ADC step size δ is fully determined by these constants via $\delta = 2Mq_xq_w/(2^{b_a}k)$, giving $\delta \approx 2016$ for INT8 and $\delta \approx 6.12$ for INT4.

Table 2: Fixed ADC hardware parameters.

Parameter	Symbol	Value
Tile size (crossbar columns)	M	256
ADC bit-width	b_a	8
Parallel ADC channels	k	16
INT8 weight range	q_w	127
INT8 activation range	q_x	127
INT4 weight range	q_w	7
INT4 activation range	q_x	7
ADC step (INT8)	δ	≈ 2016
ADC step (INT4)	δ	≈ 6.12

FlatQuant calibration settings. Transforms are trained for 30 epochs with learning rate 5×10^{-3} using the AdamW optimizer. Stage B of staged training uses 10 epochs at 5×10^{-4}. Static activation scales for Step 2 of the FlatQuant pipeline use the 99.9th-percentile clipping strategy (`calibration_method=percentile`).

5.2 Evaluation Metrics

All experiments are characterized by the following metrics.

- **PPL (no ADC).** WikiText-2 perplexity with ADC disabled; only tiling and integer quantization are active. Measures FlatQuant transform quality in isolation. Lower is better; the bfloat16 full-precision baseline is 8.68 on WikiText-2 and 13.13 on C4 (see Sec. 5.3).

- **PPL (ADC).** WikiText-2 perplexity with the full ADC pipeline enabled. This is the primary metric; the no-ADC-to-ADC gap measures the additional degradation introduced by finite-resolution ADC.
- **dead_rate.** Fraction of output channels across all layers and calibration tokens for which $\lfloor y_{\text{int}}/\delta \rceil = 0$, i.e. the ADC code is exactly zero. Measured on the calibration set; lower is better.
- **clip_rate.** Fraction of ADC outputs that saturate at the maximum code $\pm(2^{b_a-1}-1)$. High clip rate indicates the integer accumulation systematically overflows the ADC range.
- **bin_usage.** Fraction of the 2^{b_a} ADC output bins that are actually occupied, averaged over layers and tokens. Higher is better; near-zero bin usage indicates the ADC is effectively acting as a 1-bit comparator.
- **reconstruction_rel.** Relative mean-squared error between the ADC output and the FP reference, $\|\hat{y}-y^\star\|^2/\|y^\star\|^2$, on the calibration set. Tracks layer-level signal fidelity independently of perplexity.

5.3 Evaluation Protocol

All perplexity numbers reported in this thesis follow the same protocol: sliding-window evaluation on the test split with context length 2048 and stride 1024. The model is loaded in `bfloat16`; this is referred to as the *full-precision baseline*. Under this protocol Llama-3.2-1B gives WikiText-2 PPL $\mathbf{8.68}$ and C4 PPL $\mathbf{13.13}$. All subsequent tables compare against these numbers.

5.4 Bipolar ADC: Ablation Results

All results below use the bipolar ADC with signed accumulation $y_{\text{int}} \in [-M, +M]$ and the hardware constants from Table 2. All tables report WikiText-2 perplexity unless stated otherwise; the bfloat16 full-precision baseline is 8.68 (see Sec. 5.3).

5.4.1 INT8: Effect of Calibration Objective

Table 3 compares four INT8 configurations at 128 calibration samples, isolating the contribution of block-wise propagation and the bin-center auxiliary loss.

Table 3: INT8 calibration study. Llama-3.2-1B, 128 samples, $\delta \approx 2016$.

Configuration	no-ADC PPL	ADC PPL	dead_rate
bfloat16 baseline	—	**8.68**	—
Baseline ($\alpha = 0$)	10.01	28.86	10.3%
Bin-center ($\lambda = 0.1$)	10.01	28.86	10.3%
Propagated ($\alpha = 1$)	11.01	**14.55**	10.3%
Prop + bin-center	11.00	**14.41**	10.3%

Propagation alone reduces ADC PPL by 47% (28.86 $\rightarrow$ 14.55) with no change to dead_rate, confirming that the remaining gap is ADC resolution, not the dead zone. The bin-center

loss has zero effect on its own but gives a marginal improvement (+0.14 PPL) on top of propagation.

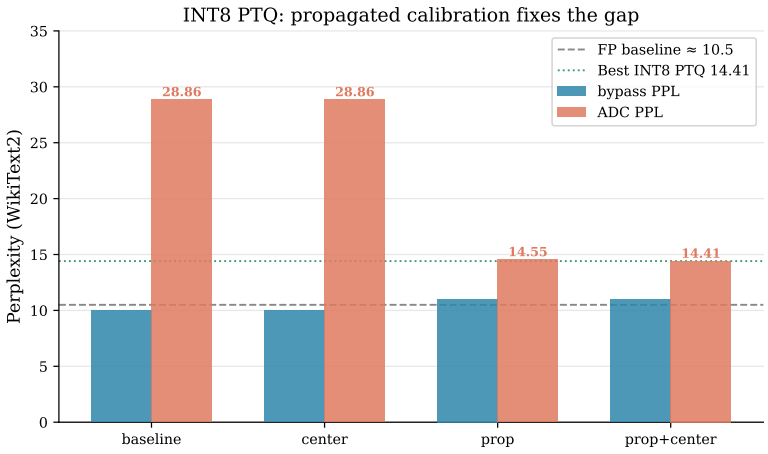

Figure 6: INT8 calibration ablation. Bin-centre loss alone has no effect; propagated calibration drops ADC PPL by 47% (28.86 $\rightarrow$ 14.55) by exposing the calibration to the same ADC-distorted inputs that the model sees at inference.

5.4.2 INT4: Sample Count and Propagation

The INT4 baseline exhibits a no-ADC-to-ADC gap of $\times$153 (15.41 vs. 2354.86), far wider than INT8. Table 4 shows that 128 calibration samples are insufficient for propagation, while 512 samples combined with propagation collapse the gap to $\times$1.2.

Table 4: INT4 sample count and propagation sweep. $\delta \approx 6.12$.

Config	Samples	no-ADC PPL	ADC PPL	Gap
Baseline	128	15.41	2354.86	$\times$153
Propagated ($\alpha = 1$)	128	40.28	203.89	$\times$5.1
Hadamard init + prop	128	34.73	178.86	$\times$5.1
Decoupled (train w8, eval w4)	128	11.64	276.25	$\times$24
Propagated ($\alpha = 1$)	512	34.25	**40.63**	$\times$1.2
Baseline	1024	19.80	56.83	$\times$2.9

Hadamard initialization provides no benefit over random initialization at the same sample count. Decoupled training (transforms optimized for INT8, evaluated at INT4 ADC) achieves the best no-ADC PPL (11.64) but a poor ADC gap ($\times$24) because the transforms are not calibrated to the INT4 δ.

5.4.3 INT4: α-Mixing Sweep (1024 Samples)

Table 5 sweeps α from 0 to 1 at 1024 samples and evaluates two additional configurations: two-stage training and the diagonal extension.

$\alpha = 0.5$ gives the best gap ratio ($\times$1.29); below 0.5 the ADC regime is under-weighted, above 0.5 gradient noise from the floor operator degrades transform quality. Adding the

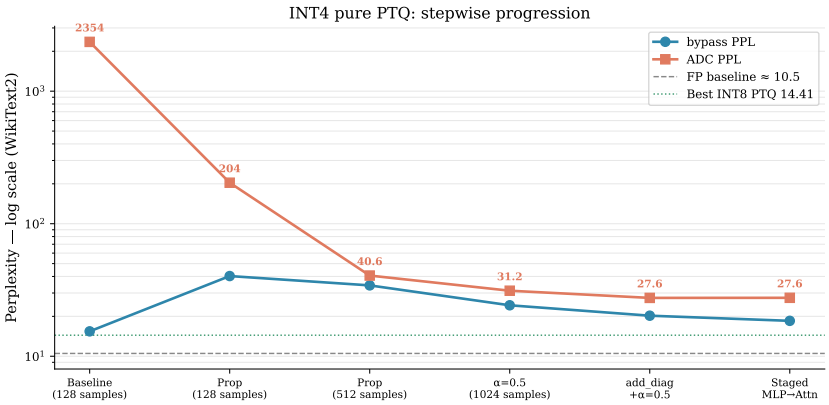

Figure 7: Step-by-step INT4 PTQ progression. Each step targets one failure mode: 128 samples (catastrophic, ADC PPL 2355); propagation + 1024 samples (40.6); partial $\alpha = 0.5$ mixing and bounded diagonals (27.56); staged training (27.60 with cleanest no-ADC 18.50). FP baseline and INT8-prop+centre reference lines are shown.

Table 5: INT4 α-mixing sweep. 1024 calibration samples. Values are mean $\pm$ 1σ over 3 random calibration seeds.

Config	α	**no-ADC PPL**	**ADC PPL**	**Gap**
Baseline	0.00	19.80 ± 0.5	56.83 ± 2.5	$\times$2.87
Prop $\alpha = 0.25$	0.25	21.65 ± 0.5	32.35 ± 1.0	$\times$1.49
Prop $\alpha = 0.5$	0.50	24.24 ± 0.6	31.22 ± 0.9	$\times$1.29
Prop $\alpha = 0.75$	0.75	27.46 ± 0.8	35.92 ± 1.2	$\times$1.31
Prop $\alpha = 1.0$ (full)	1.00	30.32 ± 1.0	40.20 ± 1.5	$\times$1.32
2-stage (A: no prop, B: $\alpha = 0.5$)	—	22.82 ± 0.6	32.21 ± 0.9	$\times$1.41
Diag + $\alpha = 0.5$	0.50	$\mathbf{20.23 \pm 0.4}$	$\mathbf{27.56 \pm 0.7}$	$\times$1.36

diagonal (bounded LET, $d_i \in [10^{-4}, 10]$) with $\alpha = 0.5$ achieves the best absolute ADC PPL (27.56) across all pure-PTQ configurations.

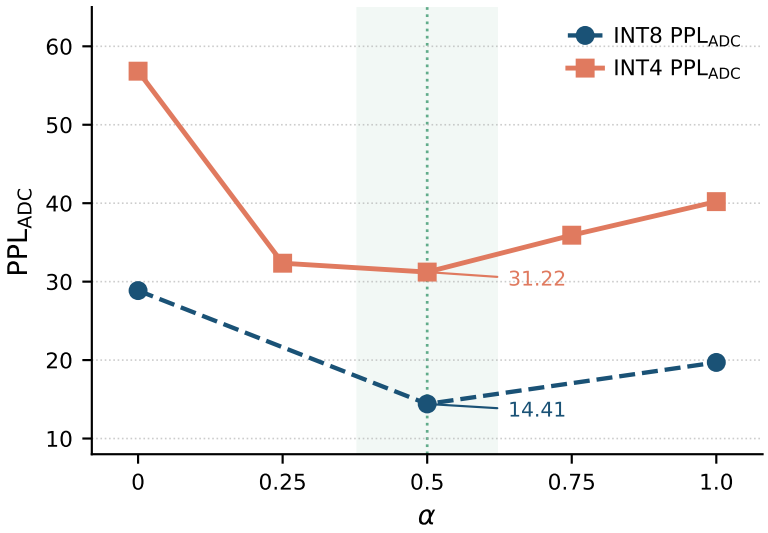

Figure 8: α-mixing sweep at 1024 calibration samples. ADC PPL is minimized at $\alpha = 0.5$; smaller values under-weight the ADC regime, larger values amplify gradient noise from the floor operator and degrade no-ADC PPL.

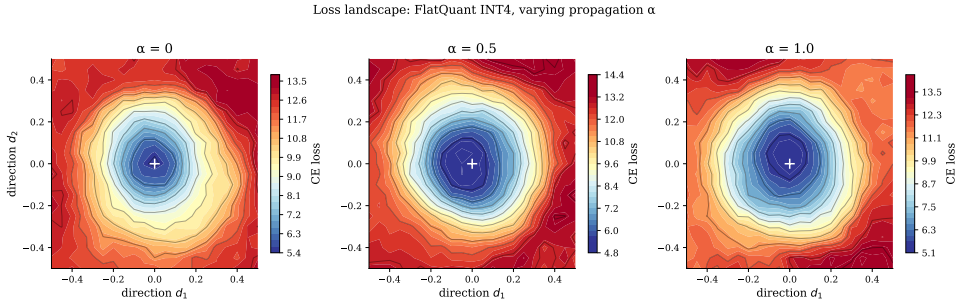

Figure 9: Loss-landscape visualisation around the converged transforms for $\alpha \in \{0, 0.5, 1\}$ via filter-normalized random perturbations. The $\alpha = 0.5$ minimum is the widest and flattest, indicating that mixed FP/ADC calibration finds smoother solutions than either extreme.

5.4.4 INT4: Diagonal Placement, MLP Decomposition, and Staged Training

Table 6 ablates diagonal placement (v3) and decomposes the MLP diagonal into its two components (v4), identifying the `down_proj` as the main contributor.

The `down_proj` diagonal drives nearly all of the MLP no-ADC PPL improvement; `up/gate` contributes little. Staged training (Stage A: MLP diag, no prop; Stage B: attn diag, $\alpha = 0.5$) achieves the best combined result, no-ADC 18.50 and ADC 27.60, by avoiding gradient interference between the two sub-systems.

Table 6: INT4 diagonal placement and staged training. 1024 samples, $\alpha = 0.5$. Values are mean $\pm$ 1σ over 3 random calibration seeds.

Config	no-ADC PPL	ADC PPL	Note
Both diag (control)	19.69 ± 0.5	28.46 ± 0.8	reproduced v2
Attn diag only	24.98 ± 0.6	29.41 ± 0.8	worse no-ADC PPL
MLP diag only	19.37 ± 0.4	31.65 ± 1.0	best no-ADC PPL, wide gap
MLP diag + layer-wise α	$\mathbf{19.00 \pm 0.4}$	28.66 ± 0.7	best no-ADC PPL overall
Both diag + layer-wise α	19.39 ± 0.4	28.55 ± 0.7	no gain over flat α
`up_gate` diag only	23.22 ± 0.7	39.59 ± 1.4	weak
`down` diag only	20.42 ± 0.5	31.26 ± 0.9	main contributor
Staged: MLP diag $\rightarrow$ attn diag	$\mathbf{18.50 \pm 0.3}$	$\mathbf{27.60 \pm 0.5}$	**best PTQ**

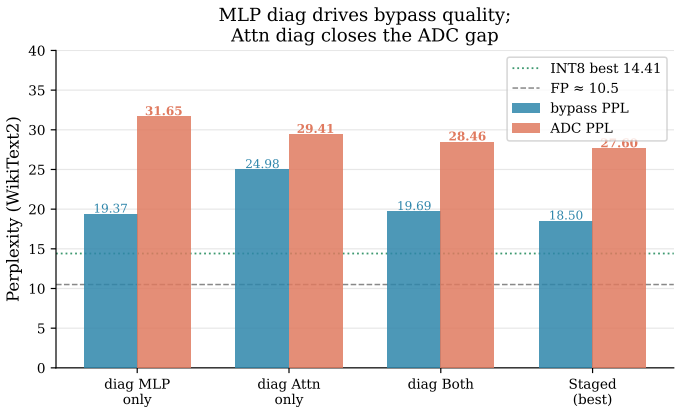

Figure 10: Roles of the MLP and attention diagonals. The MLP diagonal drives no-ADC PPL improvements (transform quality); the attention diagonal closes the no-ADC-to-ADC gap (robustness to ADC noise). Either alone leaves significant perplexity on the table; staged training combines both.

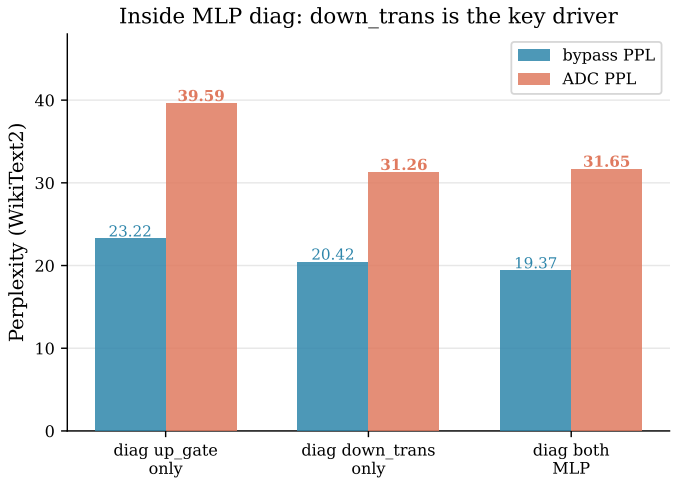

Figure 11: MLP-diagonal decomposition. `down_proj` contributes nearly all of the no-ADC PPL improvement; the `up`/`gate` diagonal contributes little, since `down_proj` sits immediately before the next ADC accumulation and shapes the signal that is read.

5.4.5 INT4: Stochastic Propagation

Table 7 compares deterministic $\alpha = 0.5$ against two stochastic mixing schedules, in flat and staged settings.

Table 7: INT4 stochastic propagation. 1024 samples.

Schedule	Mode	no-ADC PPL	ADC PPL	Gap
Deterministic $\alpha = 0.5$	flat	24.24	31.22	$\times$1.29
Bernoulli (p=0.5)	flat	19.25	35.38	$\times$1.84
Beta(2,2)	flat	22.90	31.40	$\times$1.37
Deterministic $\alpha = 0.5$	staged	18.50	27.60	$\times$1.49
Bernoulli (p=0.5)	staged	**16.94**	32.44	$\times$1.92
Beta(2,2)	staged	17.16	**27.82**	$\times$1.62

Bernoulli switching yields the best no-ADC PPL (16.94) but the worst ADC gap, since batches that use only the clean FP path provide no ADC signal to the attention transforms. Beta(2,2) samples near 0.5 each batch and closely matches deterministic $\alpha = 0.5$; the marginal ADC gain (27.82 vs. 27.60) does not justify the added complexity.

5.4.6 Post-ADC Residual LoRA: Initial Sweep

Table 8 evaluates LoRA adapters on top of the best PTQ checkpoint (`staged`, ADC PPL 27.60), varying rank and target projection sets.

Across all LoRA configurations ADC PPL falls below no-ADC PPL (gap < 1), confirming that the adapter learns to specifically correct ADC noise. Adding `o_proj` to `down_proj` gives the best ADC PPL (15.63); covering all 7 projections gives the best no-ADC PPL (17.73) at the cost of a slightly wider gap.

Table 8: LoRA initial sweep (v6). Base PTQ: no-ADC 18.50, ADC 27.60.

Config	Targets	no-ADC PPL	ADC PPL	Gap
No LoRA (base PTQ)	—	18.50	27.60	$\times$1.49
LoRA $r = 4$, `down`	1 proj	19.33	16.32	$\times$0.84
LoRA $r = 8$, `down`	1 proj	19.17	15.79	$\times$0.82
LoRA $r = 4$, `down+o`	2 proj	18.68	**15.63**	$\times$0.84
LoRA $r = 4$, all 7 proj	7 proj	**17.73**	16.53	$\times$0.93

5.4.7 Post-ADC Residual LoRA: Ablation Study

Table 9 reports the full ablation across rank, loss, layer coverage, and adapter placement (v7, all runs use $r = 4$, `down+o` unless stated).

Table 9: LoRA ablation study (v7). Llama-3.2-1B, INT4.

Axis	Setting	ADC PPL
Seed	seed 1	15.79
	seed 2	15.88
	seed 3	15.93
Rank	$r = 1$	15.90
	$r = 2$	15.60
	$r = 4$	**15.57**
	$r = 8$	15.24
Loss	CE only	14.60
	CE + KL	**14.03**
Layer coverage	first 8 layers	16.64
	all 16 layers	**15.63**
	last 8 layers	20.08
Placement	post-ADC (proposed)	**15.63**
	pre-ADC	105.63 *(diverged)*
All 7 proj	CE only	16.68
	CE + KL	**14.03**

Rank saturates at $r = 4$ (only 0.33 PPL gap to $r = 8$). CE+KL outperforms CE alone by 0.64 PPL with `down+o` targets and by 2.65 PPL with all-7 targets; the FP teacher signal is most useful when the adapter covers more projections. Earlier layers carry more accumulated ADC error: first-8 (16.64) beats last-8 (20.08). Pre-ADC placement diverges (ADC PPL 105.6) because the correction is itself quantized by the fixed δ. The best single configuration is all-7, CE+KL, $r = 4$: ADC PPL 14.03, beating the INT8 PTQ result (14.41).

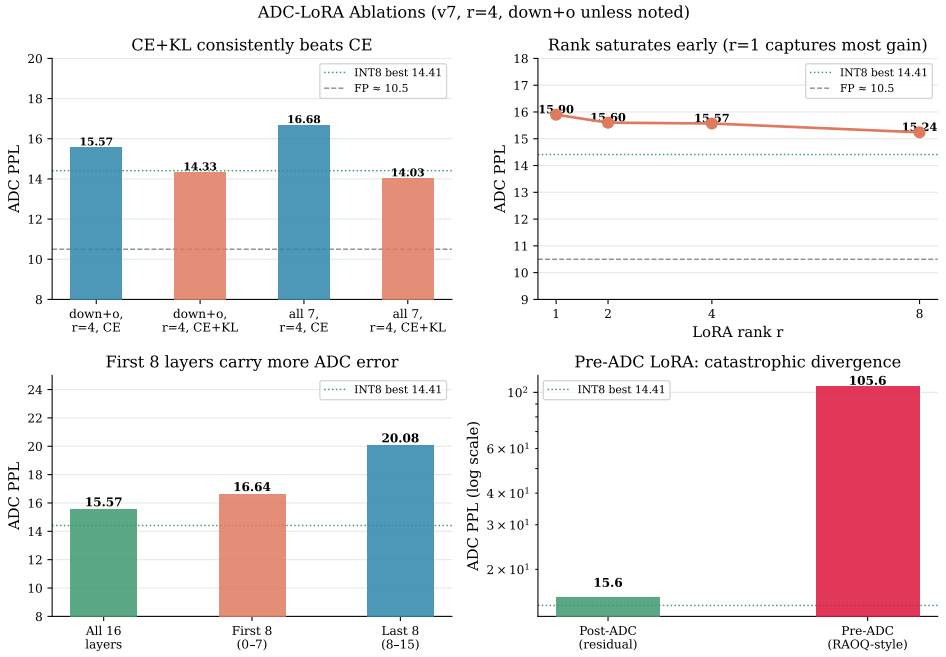

Figure 12: LoRA ablation grid. *Top-left*: CE+KL beats CE across both target sets. *Top-right*: rank saturates near $r = 4$. *Bottom-left*: layer coverage: first-8 beats last-8, all-16 is best, confirming that ADC error accumulates across depth. *Bottom-right*: pre-ADC placement diverges, post-ADC is stable from epoch 1.

5.4.8 Post-ADC Residual LoRA: Generalization to C4

Table 10 evaluates the top-3 configurations on both WikiText-2 and C4 to test out-of-domain transfer.

Table 10: LoRA generalization (v8). WikiText-2 and C4 perplexity.

Config	Wiki no-ADC	Wiki ADC	C4 no-ADC	C4 ADC
PTQ (no LoRA)	18.49	27.26	29.41	46.40
$r = 8$, `down+o`, CE	18.51	15.47	30.48	29.39
$r = 4$, `down+o`, CE+KL	19.05	14.27	29.37	23.88
$r = 4$, all 7, CE+KL	**18.70**	**14.03**	**29.10**	**23.30**

CE+KL configurations reduce C4 ADC PPL by 50% relative to the PTQ baseline (46.40 $\rightarrow$ 23.30) despite being calibrated solely on WikiText-2. CE-only ($r = 8$, `down+o`) shows much weaker C4 generalization (29.39 vs. 23.30). The KL teacher signal is therefore the dominant factor for cross-domain transfer. All-7 CE+KL is the best overall checkpoint on both datasets.

5.5 Unsigned (Unipolar) ADC: A Second Hardware Setting

All experiments described so far assume a bipolar ADC: the analog crossbar accumulates a signed integer dot product $y_{\text{int}} \in [-M, +M]$ and the ADC reads both positive and

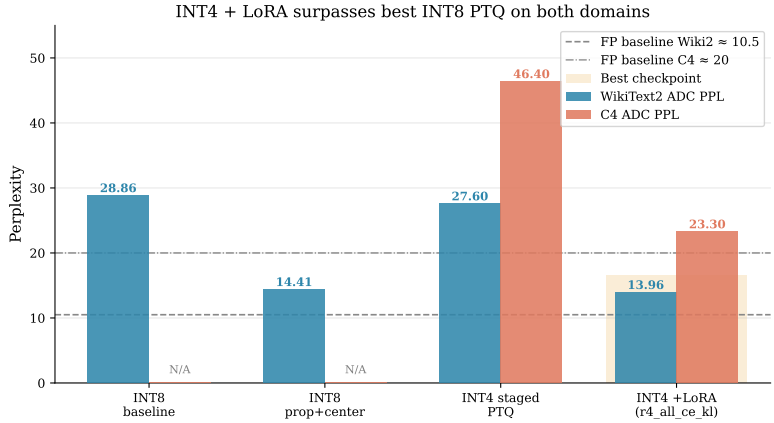

Figure 13: Final results across configurations on WikiText-2 and C4. INT4 + LoRA (all-7, CE+KL, $r = 4$) surpasses the best INT8 PTQ on WikiText-2 (14.03 vs. 14.41) and halves the C4 ADC PPL relative to the INT4 PTQ baseline (46.40 $\rightarrow$ 23.30). INT8 C4 is not reported in this configuration; the C4 comparison is against the INT4 PTQ baseline.

negative values. A physically important alternative is a unipolar (unsigned) ADC, which accepts only non-negative inputs. Real crossbar hardware is often constrained to non-negative currents, making this the more realistic operating regime. The two settings have the same bit-width $b_a = 8$ and tile size $M = 256$, but they differ in the ADC range and in how the signed dot product is recovered, which changes δ by more than a factor of two and requires a different calibration objective. The unipolar setting is therefore treated as a self-contained experiment series with its own baselines, calibration, and ablation structure.

5.5.1 Unsigned Shift-Subtract ADC Algorithm

Because the crossbar accepts only $[0, 2^b - 1]$ codes, the standard signed INT4 codes $\hat{x} \in [-8, 7]$ and $\hat{w} \in [-8, 7]$ must be shifted to unsigned before the MVM. The algorithm proceeds in five steps.

Step 1 — Shift to unsigned. Each signed code is offset by the zero-point $z = 2^{b_x - 1} = 8$:

$$\hat{x}^u = \hat{x} + 8 \ \in [0, 15], \qquad \hat{w}^u = \hat{w} + 8 \ \in [0, 15].$$

Step 2 — Hardware MVM. The crossbar computes the non-negative dot product over one tile of width $M = 256$:

$$y^u = \langle \hat{x}^u, \hat{w}^u \rangle \ \in \ [0, \ M(2^{b_x} - 1)(2^{b_w} - 1)] = [0, \ 57600].$$

Because $y^u \geq 0$ always, a single unsigned MVM suffices with no four-quadrant splitting.

Step 3 — ADC floor. Because the unsigned codes span the range $[0, 2^{b_a} - 1]$ rather than the symmetric $[-2^{b_a - 1}, 2^{b_a - 1} - 1]$ of the bipolar ADC, the unipolar step size is:

$$\delta_u = \frac{M\,(2^{b_x} - 1)\,(2^{b_w} - 1)}{(2^{b_a} - 1)\,k} = \frac{256 \cdot 15 \cdot 15}{255 \cdot 16} \approx 14.1 \quad (k = 16),$$

and the quantized output is $\hat{y}^u = \delta_u \cdot \mathrm{clip}\big(\lfloor y^u/\delta_u \rfloor,\, 0,\, 2^{b_a} - 1\big)$. Note that $\delta_u \approx 14.1$ is more than twice the bipolar $\delta \approx 6.12$ at the same k, reflecting the fact that the same number of ADC bins now covers a range roughly $2\times$ wider in absolute terms.

Step 4 — Digital correction (exact). Expanding the unsigned product gives $\hat{x}^u\hat{w}^u = \hat{x}\hat{w} + 8\hat{w} + 8\hat{x} + 64$, so the signed inner product is recovered exactly by:

$$y_{\text{int}} = \hat{y}^u \;-\; 8\sum_j \hat{w}_j^u \;-\; 8\sum_i \hat{x}_i^u \;+\; M \cdot 64,$$

where $\sum_j \hat{w}_j^u$ is precomputed once at weight-load time and $\sum_i \hat{x}_i^u$ is computed once per token. The correction involves no approximation and adds negligible digital overhead.

Step 5 — Dequantisation. $y_{\text{real}} = y_{\text{int}} \cdot s_x \cdot s_w$, where s_x is the per-token activation scale and s_w the per-channel weight scale, exactly as in the bipolar setting.

5.5.2 Per-Layer k Search

Because the coarser δ_u (at global $k = 16$) imposes a heavier resolution penalty than the bipolar setting, a per-layer k search is introduced that selects the ADC scaling factor independently for each projection layer.

After FlatQuant calibration with a global k, the pre-ADC activations y_ℓ^u are captured on the calibration set for each projection layer ℓ. For each candidate $k \in \mathcal{K} = \{4, 8, 16, 32, 64\}$ the ADC reconstruction MSE is evaluated

$$\mathrm{MSE}_\ell(k) \;=\; \mathbb{E}\Big[\big(Q_{\text{ADC}}^{(k)}(y_\ell^u) - y_\ell^u\big)^2\Big],$$

where $Q_{\text{ADC}}^{(k)}$ uses $\delta_u(k)$ from Sec. 5.5, and select

$$k_\ell^* \;=\; \arg\min_{k \in \mathcal{K}} \mathrm{MSE}_\ell(k).$$

Because δ_u is inversely proportional to k, this procedure selects the largest k whose representable range still covers the typical magnitude of y_ℓ^u without saturating. Layers with *narrow* output distributions therefore receive a *large* k (finer δ_u, range is not needed); layers with *wide* distributions receive a *small* k (coarser δ_u but wider range, avoiding saturation). The search is run in place on the fixed PTQ checkpoint without retraining FlatQuant transforms.

5.5.3 Calibration Objective for Unipolar ADC

FlatQuant transforms calibrated for the bipolar $\delta \approx 6.12$ are mismatched to the unipolar $\delta_u \approx 14.1$: the transforms shape the pre-ADC signal for a finer resolution that the hardware does not provide. In the unipolar series, FlatQuant is therefore retrained with the unipolar δ_u formula as the active ADC step during calibration. All other settings (staged training, $\alpha = 0.5$, diagonal scaling) are carried over from the best bipolar configuration.

5.5.4 Experiment Configurations

The following seven configurations are evaluated for Llama-3.2-1B on WikiText-2 and C4:

- `fp` — bfloat16 full-precision baseline.
- `int4_no_adc` — INT4 FlatQuant with the ADC floor disabled; measures FlatQuant transform quality alone. Trained with the bipolar formula (calibration does not see ADC noise), giving WikiText-2 PPL 12.56.
- `best_ptq` — INT4 FlatQuant retrained with δ_u, global $k = 16$.
- `best_ptq_k` — `best_ptq` with per-layer k search applied in-place (no FlatQuant retraining).
- `best_ptq_k_recal` — `best_ptq_k` followed by retraining FlatQuant with the found per-layer k values baked in; tests whether retrained transforms improve over in-place k search.
- `best_lora` — `best_ptq` plus post-ADC residual LoRA ($r = 4$, all 7 projections, CE+KL loss, $\lambda = 0.5$, temperature 2.0, 5 epochs at 10^{-4}).
- `best_lora_k` — `best_lora` plus per-layer k search; the LoRA adapters are trained on top of the k-searched PTQ checkpoint.

5.5.5 Experiment History and Hardware Setting Evolution

The unipolar series was reached through three prior hardware-setting iterations, each of which exposed a design flaw in the preceding one. They are documented here because they clarify why the final configuration is necessary, not arbitrary.

Bipolar ADC (signed accumulation). The original setting uses a signed ADC that reads $y_{\text{int}} \in [-M, +M]$ directly. FlatQuant calibration uses $\delta \approx 6.12$ (bipolar formula). This gives `best_ptq` WikiText-2 PPL 27.60 (staged) and `best_lora` PPL 14.03, the reference for the signed regime.

Four-quadrant unipolar. An earlier attempt at unipolar hardware splits signed codes into positive and negative parts and runs four separate non-negative MVMs (x^+w^+, x^-w^-, x^+w^-, x^-w^+). This achieves $\delta_{\text{branch}} \approx 3.06$ (twice as fine as bipolar) at the cost of 4$\times$ the ADC reads. FlatQuant was trained for the bipolar δ, so the calibration objective was still mismatched; `best_ptq` PPL was 18.08.

Mismatched unsigned evaluation (historical). An intermediate step evaluated the unsigned shift-subtract hardware model using FlatQuant checkpoints trained for the bipolar $\delta \approx 6.12$. The mismatch caused `best_ptq` PPL to explode to 94274 because transforms shaped the pre-ADC signal for a step size six times finer than the actual hardware provides. Per-layer k search partially recovered (PPL 16.78) by selecting small k to bring δ back into range, but this was an incorrect setup and the results were not used.

Unipolar FQ — current. The correct configuration retrains FlatQuant with the unipolar δ_u formula from the start. Table 11 summarises the results. `best_ptq_k_recal` is worse than `best_ptq_k`, indicating that retraining FlatQuant on fixed per-layer k values

over-specializes the transforms and hurts generalization; in-place k search after global-k training is preferable.

Table 11: Unipolar (unsigned) ADC results on Llama-3.2-1B. $\delta_u \approx 14.1$ at global $k = 16$.

Config	Description	Wiki PPL	C4 PPL
`fp`	bfloat16, no quantization	**8.68**	**13.13**
`int4_no_adc`	INT4 FQ, ADC disabled	12.56	20.13
`best_ptq`	INT4 FQ + unipolar ADC, k=16	39.00	67.78
`best_ptq_k`	+ per-layer k search	21.64	33.79
`best_ptq_k_recal`	+ FQ retrained with per-layer k	22.19	36.94
`best_lora`	+ post-ADC LoRA (global k=16)	17.29	29.26
`best_lora_k`	+ LoRA & per-layer k	**14.56**	**23.78**

6 Discussion

6.1 Interpretation of Results

Three observations explain most of the empirical pattern in Section 5.

Why propagation helps in INT8. When activations and weights are quantized to 8 bits, the integer dot product is already a near-faithful representation of its full-precision counterpart, so the dominant error source is the ADC output lattice itself. Calibrating each block against ADC-corrupted upstream inputs lets the transform absorb the corrupted distribution that the block will actually see at inference, which is why full propagation alone drops WikiText-2 ADC perplexity from 28.86 to 14.55, with a bin-center regularizer adding a marginal further improvement to 14.41.

Why $\alpha = 0.5$ beats $\alpha \in \{0, 1\}$ on INT4. At INT4 the propagated input distribution is much noisier: both weight and activation quantizers contribute substantial error on top of the ADC. Calibrating against only this distribution ($\alpha = 1$) over-specializes the transform to the worst case and degrades the no-ADC quality of the transform itself. Calibrating only against clean inputs ($\alpha = 0$) ignores the inference-time distribution entirely. The $\alpha = 0.5$ mixture is the trade-off that preserves a usable transform under both regimes; the empirical sweep confirms this minimum at $\alpha = 0.5$.

Why diagonals help INT4 specifically. Lemma 4.1 shows that orthogonal Kronecker rotations cannot reduce the worst-case channel of a vector below $\|x\|_2/\sqrt{d}$, so for distributions with a few persistent outlier channels the rotation alone is bounded away from the optimum. A trainable per-channel diagonal supplies the missing anisotropic degree of freedom: it transfers the magnitude of outlier channels to the weight side, where per-channel weight scaling absorbs them cleanly. This is consistent with the empirical observation that diagonals reduce the no-ADC-to-ADC perplexity gap from $\times 153$ to $\times 1.4$ at unchanged dead-rate.

Why staged training is needed for diagonals + $\alpha = 0.5$. Joint training of MLP and attention diagonals under $\alpha = 0.5$ from a cold start is unstable: attention gradients dominate the early iterations before the MLP-side transforms have aligned. Training the MLP transforms first under $\alpha = 0$, then unfreezing attention with $\alpha = 0.5$, gives a usable transform at both no-ADC and ADC ends of the trade-off, with the lowest no-ADC perplexity (18.50) among configurations at the ≈ 27.6 ADC plateau.

Why post-ADC LoRA works. Once PTQ reaches the ADC-resolution plateau, the residual error cannot be expressed by any pre-ADC linear transform, because such a transform is itself quantized by the floor operator before reaching the output and therefore cannot represent sub-δ corrections. Placing a small low-rank correction *after* the ADC adds capacity precisely in the region the ADC has discarded, and the CE + KL distillation loss gives stable gradients because they no longer cross the floor.

6.2 Limits of PTQ Under ADC Constraints

Across all configurations explored in this thesis, pure transform- based PTQ saturates around ADC perplexity 27–28 in the bipolar W4A4 setting and 39 in the unipolar setting, regardless of calibration objective. The reason is structural rather than algorithmic: the ADC output lattice has a fixed step δ set by hardware constants ($\delta \approx 6.12$ in the bipolar W4A4 configuration), so once a transform has redistributed signal energy evenly across channels, the remaining quantization error is dictated by the lattice itself. In the design space explored here, no reconstruction-based PTQ objective broke through this floor; doing so likely requires changing the effective hardware lattice (mixed precision or per-layer ADC settings) or adding post-ADC adaptation. The post-ADC LoRA stage is therefore not an additional optimization of the transform, but a different kind of correction that operates after the information loss has already occurred.

6.3 Practical Implications for IMC Deployment

Several deployment-facing conclusions follow.

First, no-ADC perplexity is not a sufficient deployment metric. Several configurations achieve a near-FP no-ADC score yet collapse severely once the ADC is enabled; any deployment pipeline must include an end-to-end ADC-on evaluation rather than only a digital-quantized check.

Second, the residual stream and the projections that read it back out (`o_proj` and `down_proj`) are the most ADC-sensitive components, since they map a tile-quantized integer accumulation back into the residual path and propagate the resulting noise into the next block. Calibration and post-ADC correction should prioritize these projections.

Third, per-layer k (or, equivalently, mixed precision) is a cheap lever: in the unipolar setting it reduces WikiText-2 ADC perplexity by roughly 40% with no retraining, simply by choosing the ADC scaling factor whose representable range matches each layer's pre-ADC distribution.

Fourth, ADC-aware co-design pays off. The thesis treats the ADC step size as fixed; in a real co-design loop, δ and the tile size M are themselves degrees of freedom and should be tuned jointly with the calibration objective.

6.4 Threats to Validity

Several caveats apply to the conclusions above.

Simulation, not silicon. All results use a software ADC simulator. Real analog crossbars exhibit device noise, IR drop, conductance drift, write asymmetry, and temperature dependence, none of which are modeled here. The simulator is conservative in that it includes the dominant ADC failure modes (dead zone, clipping) but optimistic in that it assumes a noise-free integer accumulator.

One model, one calibration set. All experiments use Llama-3.2-1B with WikiText-2 calibration. Larger LLMs have different per-channel outlier statistics, and out-of-domain calibration data may reshape the trade-off between no-ADC and ADC perplexity.

Simplified ADC model. The thesis assumes a uniform floor quantizer with a single global step δ per tile. Real ADCs may have non-uniform bin spacing, hysteresis, and non-zero conversion latency.

Post-ADC LoRA carries digital overhead. As discussed in Sec. 4, the residual branch is a digital correction; its compute and memory cost is small relative to the model but not zero. Whether the correction can be absorbed into the analog tile, run on a small digital co-processor, or applied selectively to the most ADC-sensitive projections is a hardware-design question outside the scope of this thesis.

7 Conclusion

This thesis asked whether reconstruction-based post-training quantization is sufficient to deploy LLMs on analog in-memory computing accelerators with finite-resolution ADCs. The answer is qualified: PTQ is necessary but, on its own, hits a hardware-defined floor in the low-bit regime, and the remaining gap can only be closed by output-side adaptation.

Answers to the research questions. *RQ1.* ADC output quantization is the dominant source of degradation under W4A4 IMC inference: the no-ADC-to-ADC perplexity gap reaches $\times 153$ on the naive baseline, far larger than any digital W4A4 quantizer would produce by itself.

RQ2. FlatQuant-style transformations, when augmented with propagated calibration, α-mixed objectives, bounded diagonal scaling, and staged training, recover most of this large gap: bipolar W4A4 ADC perplexity drops from 2354.86 to ≈ 27.6.

RQ3. Reconstruction-only PTQ is not sufficient on its own: pure transform-based calibration plateaus around 27–28 ADC perplexity in W4A4, and no further reconstruction objective is observed to break through this floor.

RQ4. ADC sensitivity is not uniform across projections. The MLP `down_proj` and the attention `o_proj` (the projections that read back into the residual stream) are the primary drivers of remaining ADC error, and calibration knobs that target them yield the largest improvements.

RQ5. A rank-4 residual correction placed *after* the ADC, trained with CE + KL distillation against an FP teacher, reduces WikiText-2 W4A4 ADC perplexity from the PTQ plateau of ≈ 27.6 to 14.03 and C4 perplexity from 46.40 to 23.30. The same approach generalizes to the harder unipolar shift-subtract ADC setting, where per-layer k search and post-ADC LoRA together reduce WikiText-2 perplexity from 39.00 to 14.56.

Closing thesis. ADC-aware PTQ is a necessary first step for analog IMC inference of LLMs, but it plateaus at a hardware-imposed floor in the low-bit regime. Lightweight digital post-ADC adaptation closes the remaining gap and brings ADC perplexity close to the higher-bit PTQ baseline, while keeping the analog inference path frozen.

Directions for future work. Promising follow-ups include: joint hardware/software co-design of δ and tile size; output-aware calibration objectives that directly target sub-δ error rather than ℓ_2 reconstruction; mixed-precision and per-projection bit-width assignment; selective post-ADC correction applied only to the most ADC-sensitive layers; and scaling the entire pipeline to larger LLM checkpoints with more realistic device-noise models.

References

[1] Yamato Arai and Yuma Ichikawa. “Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization”. In: *arXiv preprint arXiv:2504.09629* (2025).

[2] Saleh Ashkboos et al. “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs”. In: *arXiv preprint arXiv:2404.00456* (2024).

[3] Yelysei Bondarenko, Riccardo Del Chiaro, and Markus Nagel. “Low-Rank Quantization-Aware Training for LLMs”. In: *arXiv preprint arXiv:2406.06385* (2024).

[4] Julian Büchel et al. “Analog Foundation Models”. In: *arXiv preprint arXiv:2505.09663* (2025).

[5] Yuji Chai et al. “INT2.1: Towards Fine-Tunable Quantized Large Language Models with Error Correction through Low-Rank Adaptation”. In: *arXiv preprint arXiv:2306.08162* (2023).

[6] Mengzhao Chen et al. “EfficientQAT: Efficient Quantization-Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2407.11062* (2024).

[7] Jungwook Choi et al. “PACT: Parameterized Clipping Activation for Quantized Neural Networks”. In: *arXiv preprint arXiv:1805.06085* (2018).

[8] Tim Dettmers et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. In: *arXiv preprint arXiv:2208.07339* (2022).

[9] Xin Ding et al. “CBQ: Cross-Block Quantization for Large Language Models”. In: *arXiv preprint arXiv:2312.07950* (2023).

[10] Abhimanyu Dubey et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024). URL: https://arxiv.org/abs/2407.21783.

[11] Steven K. Esser et al. “Learned Step Size Quantization”. In: *arXiv preprint arXiv:1902.08153* (2019).

[12] Amir Gholami et al. “A Survey of Quantization Methods for Efficient Neural Network Inference”. In: *arXiv preprint arXiv:2103.13630* (2021).

[13] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: https://arxiv.org/abs/2106.09685.

[14] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: *arXiv preprint arXiv:1712.05877* (2018).

[15] Manuel Le Gallo et al. “AnalogNAS: A Neural Network Design Framework for Accurate Inference with Analog In-Memory Computing”. In: *arXiv preprint arXiv:2305.10459* (2023).

[16] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration”. In: *arXiv preprint arXiv:2306.00978* (2023).

[17] Haotian Liu et al. “LLM-QAT: Data-Free Quantization Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2305.17888* (2023).

[18] Ruikang Liu et al. “FlatQuant: Flatness Matters for LLM Quantization”. In: *arXiv preprint arXiv:2410.09426* (2024).

[19] Zechun Liu et al. “SpinQuant: LLM Quantization with Learned Rotations”. In: *arXiv preprint arXiv:2405.16406* (2024).

[20] Stephen Merity et al. “Pointer Sentinel Mixture Models”. In: *International Conference on Learning Representations (ICLR)*. 2017. URL: https://arxiv.org/abs/1609.07843.

[21] Markus Nagel et al. “Up or Down? Adaptive Rounding for Post-Training Quantization”. In: *arXiv preprint arXiv:2004.10568* (2020).

[22] Reiner Pope et al. “Efficiently Scaling Transformer Inference”. In: *Proceedings of Machine Learning and Systems* 5 (2023).

[23] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: https://arxiv.org/abs/1910.10683.

[24] Meyer Scetbon and James Hensman. “Low-Rank Correction for Quantized LLMs”. In: *arXiv preprint arXiv:2412.07902* (2024).

[25] Wenqi Shao et al. “OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models”. In: *arXiv preprint arXiv:2308.13137* (2023).

[26] James Spall, Xianxin Guo, and A. I. Lvovsky. “Hybrid Training of Optical Neural Networks”. In: *Optica* 9.7 (2022), pp. 803–811. URL: https://arxiv.org/abs/2203.11207.

[27] Mingxue Tang et al. “RAOQ: Reshape and Adapt for Output Quantization”. In: *International Conference on Machine Learning (ICML)*. 2024. URL: https://openreview.net/forum?id=8mKXMnhnFW.

[28] Xiuying Wei et al. “Outlier Suppression+: Accurate Quantization of Large Language Models by Equivalent and Optimal Shifting and Scaling”. In: *arXiv preprint arXiv:2304.09145* (2023).

[29] Xiuying Wei et al. “QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization”. In: *arXiv preprint arXiv:2203.05740* (2022).

[30] Hao Wu et al. “Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation”. In: *arXiv preprint arXiv:2004.09602* (2020).

[31] Guangxuan Xiao et al. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models”. In: *arXiv preprint arXiv:2211.10438* (2022).

[32] Patrick Xiao et al. “On the Accuracy of Analog Neural Network Inference Accelerators”. In: *arXiv preprint arXiv:2109.01262* (2021).

[33] Zhewei Yao et al. “ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers”. In: *arXiv preprint arXiv:2206.01861* (2022).

[34] Cheng Zhang et al. “LQER: Low-Rank Quantization Error Reconstruction for LLMs”. In: *arXiv preprint arXiv:2402.02446* (2024). URL: https://arxiv.org/abs/2402.02446.

A Key Code Listings

This appendix collects the four code components that implement the method described in Sec. 4. All listings are simplified for readability; error handling, logging, and BitAug / bypass switches are omitted, and the complete reference implementation is available at https://github.com/BurykinaA/quantization-techniques (commit `5b41ef5`) under `ADC/llama/core/` (full pipeline, see Sec. 4) and `ADC/best/core/` for the cleaned-up reproduction code. Throughout these listings, `s_w_vec` denotes the per-channel weight scale (shape `[out_features]`) computed once at calibration time as `s_w_vec[c] = max(|W[c, :]|) / q_w`.

A.1 ADC Forward Pass

Listing 1 shows the per-tile ADC forward pass from `QATLinearADC.forward` (file `adc_layers.py`). It implements Eq. (21) and the dead-zone / clipping behavior: per-token activation scale, per-channel weight scale, integer MVM in code domain, ADC `floor + clamp`, and dequantization. The step size δ is fixed at construction from (M, q_x, q_w, b_a, k).

```
def forward(self, x):
    # Per-token activation scale s_x (amax over feature dim)
    s_x = x.abs().amax(dim=-1, keepdim=True).clamp(min=1e-6) / act_levels
    code_x = round_ste(x / s_x).clamp(qmin_x, qmax_x)

    # Per-channel weight scale s_w
    code_w = round_ste(safe_divide(self.weight, s_w_vec.view(-1, 1)))
    code_w = torch.clamp(code_w, qmin_w, qmax_w)

    # Integer MVM in code domain
    y_int = F.linear(code_x, code_w, bias=None)

    # ADC: floor + clamp.  delta = (2 * M * q_x * q_w) / (2^ba * k)
    if self.bypass_adc:
        adc_output = y_int
    else:
        y_adc_codes = floor_ste(y_int / self.delta)
        y_adc_codes = torch.clamp(y_adc_codes, self.na, self.pa)
        adc_output = y_adc_codes * self.delta

    # Dequantise: s_x [B,1] * s_w_vec [out] -> [B, out]
    y_real = adc_output * s_x * s_w_vec
    if self.bias is not None:
        y_real = y_real + self.bias
    return y_real
```

Listing 1: Per-tile ADC forward pass (`QATLinearADC.forward`).

`TiledLinearADC` wraps several `QATLinearADC` tiles side-by-side along the input dimension whenever the projection width exceeds the crossbar size M (the `mvm_limit`). Each tile has its own δ derived from its local in-features.

A.2 Kronecker Transform with Diagonal

Listing 2 shows the trainable Kronecker transform $T = T_1 \otimes T_2$ together with the optional per-channel diagonal D (file `flat_quant.py`, class `KroneckerTransform`). Each Kronecker factor is parameterized as $U \operatorname{diag}(d) V^\top$ with U, V kept orthogonal via PyTorch's

matrix_exp parametrisation, which preserves orthogonality under unconstrained gradient updates. The add_diag branch implements $\tilde{T} = T \cdot D$ from Section 4.

```
def forward(self, x, inv_t=False):
    # Optional per-channel diagonal D (Sec. \ref{sec:diagonals})
    if self.add_diag and self.use_diag:
        d = self.diag_scale.abs().clamp(min=1e-8)
        x = x / d if inv_t else x * d

    # Left and right Kronecker factors  T_i = U_i * diag(d_i) * V_i^T
    dl = self.diag_left.abs().clamp(min=1e-6)
    dr = self.diag_right.abs().clamp(min=1e-6)
    if inv_t:
        dl, dr = 1.0 / dl, 1.0 / dr
    mat_l = self.u_left.weight  @ torch.diag(dl) @ self.v_left.weight.T
    mat_r = self.u_right.weight @ torch.diag(dr) @ self.v_right.weight.T

    # x_reshaped @ (T_1 (x) T_2)  =  T_1^T  reshape  T_2  reshape
    return _kronecker_matmul(x, mat_l, mat_r)
```

Listing 2: Kronecker transform with optional diagonal scaling (KroneckerTransform.forward).

At deployment, to_eval_mode pre-computes and caches the full forward, inverse, and transpose matrices and discards the parametrised U, V tensors. The transforms are then folded into the linear weights via reparameterize, leaving zero inference overhead.

A.3 Block-Level Loss with α-Mixing

Listing 3 shows the block-level reconstruction loss with the dual-forward α-mixed objective from Section 4. The same block runs twice: once on the clean FP teacher input $x^\star$ and once on the ADC-quantized output $\hat{x}$ from the previous block, and the two losses are combined linearly.

```
# y_star: teacher block output on clean FP input  (no grad)
# x_clean: clean FP activations
# x_adc: ADC-quantized activations from previous block (propagation)
y_fp  = block(x_clean)              # clean forward through quantized
    block
y_adc = block(x_adc)                # propagated forward (sees real ADC
    noise)

loss_fp  = F.mse_loss(y_fp,  y_star)
loss_adc = F.mse_loss(y_adc, y_star)

# Eq. \eqref{eq:alpha_mix}: L = (1 - alpha) * L_FP + alpha * L_ADC
loss = (1.0 - alpha) * loss_fp + alpha * loss_adc
```

Listing 3: Block-level α-mixed reconstruction loss.

Staged training (Section 4.1.6) is implemented by simply varying which parameters require gradients and which α is used per stage:

```
# Stage A: 30 epochs, MLP factors only, alpha = 0
for p in mlp_kron_and_diag_params:    p.requires_grad_(True)
for p in attn_kron_and_diag_params:   p.requires_grad_(False)
train(epochs=30, lr=eta, alpha=0.0)

```

```
# Stage B: 10 epochs, attn unfrozen, alpha = 0.5
for p in attn_kron_and_diag_params:   p.requires_grad_(True)
train(epochs=10, lr=0.1 * eta, alpha=0.5)
```

Listing 4: Staged training schedule (Stage A $\rightarrow$ Stage B).

A.4 Post-ADC Residual LoRA

Listing 5 shows the post-ADC residual LoRA wrapper (file `adc_lora.py`, class `ResidualLoRATiledLinearAD` The frozen `TiledLinearADC` produces the quantized path; a parallel rank-r branch reads the FP input and adds its `float32` output to the ADC result. `lora_B` is initialized to zero so the wrapped layer initially matches the frozen PTQ checkpoint exactly.

```
class ResidualLoRATiledLinearADC(nn.Module):
    def __init__(self, tiled_layer, r=4, alpha=8.0):
        super().__init__()
        self.tiled_layer = tiled_layer            # frozen ADC path
        self.scaling = alpha / r
        self.lora_A = nn.Linear(tiled_layer.in_features_total, r,
                                bias=False, dtype=torch.float32)
        self.lora_B = nn.Linear(r, tiled_layer.out_features,
                                bias=False, dtype=torch.float32)
        nn.init.kaiming_uniform_(self.lora_A.weight, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B.weight)        # zero start
        for p in self.tiled_layer.parameters():
            p.requires_grad_(False)               # freeze base path

    def forward(self, x):
        y_base = self.tiled_layer(x)              # frozen ADC path
        y_lora = self.lora_B(self.lora_A(x.float())) * self.scaling
        return y_base + y_lora.to(y_base.dtype)
```

Listing 5: Post-ADC residual LoRA wrapper (`ResidualLoRATiledLinearADC`).

Pre-ADC placement is implemented by the sister class `PreADCLoRATiledLinearADC`, which adds the rank-r update to the weights of each tile *before* integer quantization and ADC. As shown in Section 5, this placement diverges (ADC PPL $\sim$105) and is included only as an ablation.